

Algorithm Engineering

Prof. Dr. Näher

Inhaltsverzeichnis

1 Überblick **1**
- 1.1 Aspekte . . . 1
- 1.2 Literatur . . . 2

2 Datentyp **3**
- 2.1 Spezifikation . . . 3
- 2.2 Definition eines Datentyps in einer objektorientierten Programmiersprache . . . 7
- 2.3 Operationen → Methoden/Memberfunktionen . . . 7
- 2.4 Trennung von Spezifikation (Definition) und Implementierung . . . 9
- 2.5 Integer-Stack: dynamisch . . . 10
  - 2.5.1 Zwei Implementierungen . . . 10
    - I) Array-Datenstruktur . . . 10
    - Einschub: Variablen, Konstruktoren, Wertzuweisungen (C++ ↔ Java) . . . 12
    - II) Einfach verkettete Liste . . . 14
  - 2.5.2 Korrektheit einer Implementierung . . . 14

3 Vererbung ↔ Templates **16**
- 3.1 Allgemeiner: Ableitungsbaum . . . 18
  - 3.1.1 Typverträglichkeit . . . 18
- 3.2 Polymorphe Datenstrukturen . . . 19
- 3.3 Weitere Anwendungen von Vererbung . . . 23
  - 3.3.1 Generische Datenstrukturen . . . 23
    - Einfach verkettete Listen . . . 23
    - Templates oder Schablonen . . . 27

4 Fortgeschrittene Datenstrukturen und Algorithmen **30**
- 4.1 Spezifikation von Datentypen in LEDA . . . 30
  - 4.1.1 Item-Konzept . . . 30
    - Anwendung auf kürzeste Wege Algorithmen . . . 33
- 4.2 Graphen und Graphalgorithmen (in LEDA) . . . 35
  - 4.2.1 Operationen auf einem Graphen G . . . 35
    - Test, ob G azyklisch . . . 36

4.2.2 Basis-Graphtyp graph 38

5 Netzwerkflussprobleme **43**
5.1 MaxFlow-Problem 43
5.1.1 Formale Definition 43
5.1.2 Das Restnetzwerk 45
Definition 45
5.1.3 Pfaderhöhung: (Maxflow) s, t 50
Labeling-Algorithmus 50

6 Algorithmische Geometrie **53**
6.1 Operationen 53
6.2 Orientierung 54
6.2.1 Lineare Algebra 54
6.2.2 Minimum / Maximum in xy-Ordnung 54
6.2.3 Vergleich von Entfernungen 55
6.2.4 Anwendung von orientation (→ Schnitt-Test) 55
6.2.5 Problem: Konvexe Hülle 56
Einfacher Algorithmus: Gift-Wrapping 57
Inkrementeller Algorithmus 59
Divide & Conquer 62

1 Überblick

Das Gebiet der Algorithmen und Datenstrukturen wird traditionell der theoretischen Informatik zugeordnet. Die hier entworfenen und untersuchten Algorithmen verwenden häufig komplizierte Datenstrukturen und haben selten einen direkten Bezug zur Praxis, wo die asymptotische Komplexität der Probleme und Algorithmen und das Worst-case-Verhalten selten eine Rolle spielt. Hier sind häufig andere Aspekte, wie z. B. die effiziente Nutzung des Caches oder die leichte Implementier- und Wartbarkeit viel wichtiger. Algorithm Engineering versucht die Kluft zwischen Theorie und Praxis zu schließen. Entsprechend werden wir in dieser Vorlesung Algorithmen und Datenstrukturen betrachten, die in der Praxis tatsächlich Verwendung finden.

1.1 Aspekte

- **Korrektheit:** formal $\leftrightarrow$ konkretes Programm
  formal: theoretisch arbeitet man mit Invarianten (bzw. Beweistechniken)
  konkretes Programm: Hier ist eine andere Korrektheit gemeint, als bei der formalen. Es gibt viele Fehlerquellen, die schwierig zu erfassen sind. Durchs Testen des Programms können diese gefunden werden.

  In dieser Vorlesung wird es darum gehen, wie man möglichst korrekt vom Algorithmus zum Programm kommt.

- **Effizienz:** $\mathcal{O}$-Notation $\leftrightarrow$ praktische Laufzeit
  $\mathcal{O}$-Notation: Hier geht es zumeist um den Worst-Case. Konstante Faktoren werden zudem unterschlagen.
  praktische Laufzeit: Der Worst-Case tritt in der Praxis -wenn überhaupt- selten ein.

- **Software-Bibliotheken** (z.B. LEDA, Library of Efficient Data Types and Algorithms)sorgen für Vereinfachungen des Übergangs Algorithmen $\rightarrow$ Programm

- **Bausteine:** Datentyp
  Der Datentyp beschreibt eher die Funktion und weniger die konkrete Implementierung im Gegensatz zur Datenstruktur.

- **Gebiete:**
  - Grundlegende Datenstrukturen (Queue, Stack, Dictionairy, Priority Queue, ...)
  - Graphen und Netzwerke (kürzeste Wege, Flüsse)
  - Algorithmische Geometrie

1.2 Literatur

- Mehlhorn, Näher: LEDA, a platform for Combinatorial and Geometric Computing
- C++

2 Datentyp

Ein (abstrakter) Datentyp ($\rightarrow$ ADT) besteht aus einer Menge von Objekten (Werte bzw. Zahlen oder Punkte in der Geometrie) oder Zuständen und einer Menge/Familie von Operationen auf dieser Menge.
Eine Operation arbeitet auf einem Objekt/Zustand des Datentyps eventuell in Abhängigkeit weiterer Parameter, verändert es und liefert ein Ergebnis.

Beispiel 2.1

Datentyp	**Objekte**	**Operationen**
Integer	Ganze Zahlen ($\mathbb{Z}$)	$+, -, *, \backslash, +=, --, ++$
Stack<T>	Ein Keller von Werten vom Typ T	push, pop, top, empty
Getränkeautomat	Interne Zustände	Wirf 1€ ein und wähle Getränk ...
Stadtplan	Beschrifteter Graph	Neuer Knoten/Kante, Kürzester Weg, Zeichne

2.1 Spezifikation

Wie definiert man einen Datentyp?

- Name des Typs
- Menge der Objekte/Zustände
- Menge der Operationen
  Eine einzelne Operation wird definiert:
  - Syntax:
    - Name
    - Ein- und Ausgabetypen
  - Semantik:
    - eigentliche Funktion

Beispiel 2.2

1) Datentyp der ganzen Zahlen

Name: Integer

Objekte: $\mathbb{Z}$

Operationen: Add, Sub, Create, Gleich, ...

Add: integer $\times$ integer $\rightarrow$ integer
Semantik: Ordne jedem Paar (x, y) die Summe $x + y$ zu

Sub: integer $\times$ integer $\rightarrow$ integer
Semantik: Ordne jedem Paar (x, y) die Differenz $x - y$ zu

Gleich: integer $\times$ integer $\rightarrow$ integer
Semantik: Liefert true, falls die Parameter gleich sind, ansonsten false

Create: $\rightarrow$ integer (Konstruktor)
Semantik: liefert die ganze Zahl 0

2) Der Datentyp stack<T>

Name: stack<T>

Objekte/Zustände: Folge von Werten vom Typ T (bzw. $w \in T^*$), eventuell leer.

Operationen: create, push, pop, top, empty, size

create: $\rightarrow$ stack<T>
Semantik: Erzeugt den leeren Stack, d.h. die Folge der Länge 0.

push: stack<T> $\times$ T $\rightarrow$ stack<T>
Semantik: Fügt zweites Argument an erstes Argument der Folge an.

pop: stack<T> $\rightsquigarrow$ stack<T> $\times$ T
Semantik: Entfernt letztes (oberstes) Element der Folge und liefert es zurück
Precondition: Stack ist nicht leer (daher partielle Funktion)

3) Getränkeautomat

Automat akzeptiert 1€-Münzen

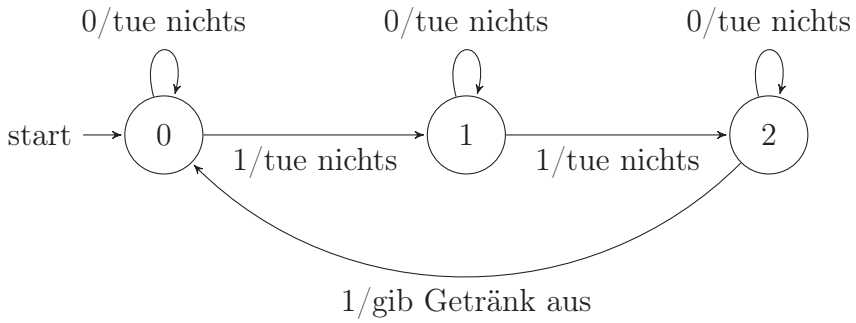
Ein Getränk kostet 3€

Operationen: Init (Reset), Akzeptiere 1€

Init $\rightarrow$ Zustand Semantik: Automat geht in Zustand 0

Akzeptiere_Euro: Zustand $\times\{0,1\} \rightarrow$ Zustand $\times$ {tue nichts, gebe Getränk aus}

Semantik: Beschreibung durch endlichen Automaten



Bemerkungen:

- Operationen können als partielle Funktion definiert sein. Man gibt dann den Definitionsbereich oft in einer Vorbedingung (Precondition) an.
  Beispiel: Stack und die Pop-Operation mit der Bedingung der Nicht-Leerheit des Stacks

- Operationen, bei der der Datentyp selbst auf der linken Seite nicht vorkommt, heißen Konstruktoren. Sie erzeugen eine neues Objekt (bzw. versetzen den Typ in einen bestimmten Zustand).
  Beispiele: create: $\rightarrow$ stack<T>
  create: int $\rightarrow$ Vektor (einer bestimmten Dimension)
  create int $\rightarrow$ stack <T> (Maximalgröße)

- Objekt- und Zustandssicht sind beide nützlich.
  - Stack/Getränkeautomat: Haben beide einen internen Zustand. Operationen können diesen ändern.
  - Integer: Hier ist die Objektsicht besser, da hier Operationen (z.B. ADD: integer × integer → integer) neue Objekte erzeugen und dabei existierende nicht geändert werden.
  - Syntax mit Parametern
    stack<T> ist ein sogenannter parametrisierter Datentyp, ein Stack mit Elementen vom Typ T (eventuell gibt es Anforderungen an den Typ T z.B. dictionary<T>: T muss eventuell linear geordnet sein ($x \leq y$)). Ein Stack hingegen braucht z.B. keine speziellen Anforderungen.
- Man kann nun eigentlich schon programmieren, obwohl über die Implementierung noch nichts bekannt ist.

Anwendung von Stack<int>

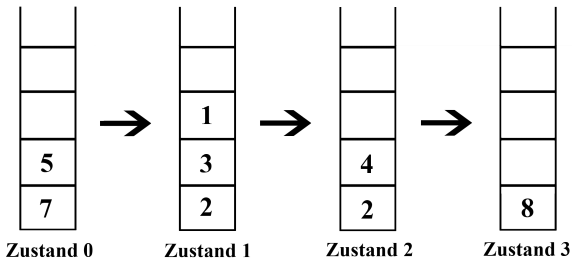
Auswertung von Postfix-Ausdrücken.
Vereinfachung:

- alle Operationen binär $+, -, *, \backslash$ (kein unäres Minus)
- Eingabe nur Zahlen 0,... ,9

Beispiel 2.3

$(7-5)*(3+1) \rightarrow 75-31+*$ (polnische Notation) Bei Stacks ist dies gut machbar, da immer nur die zwei obersten Elemente verwendet werden.

Übung: Schreiben Sie ein Programm, das diese Ausdrücke berechnet mit dem Datentyp stack<int>.



2.2 Definition eines Datentyps in einer objektorientierten Programmiersprache

Im Folgenden wurde C++ als objektorientierten Programmiersprache gewählt.

class Typename{
// Die Definition der Menge der Objekte/Zustände (als Kommentar)

private: // Deklaration von Variablen zur Darstellung der Objekte/Zustände

public: Operation 1
...
Operation k

// Kommentar: Weitere Angaben z.B. über Effizienz

2.3 Operationen → Methoden/Memberfunktionen

Syntax: Ergebnistyp Name(Arg 1, Arg 2, ..., Arg k);
z.B. void push(int x);
int pop();

Spezielle Methoden (create → Konstruktoren) haben keinen Ergebnisty.
Name = Typname
z.B. stack();
stack(int size);

in C++: Destruktoren
syntax ∼Typename();
z.B. ∼stack(); → Speicherfreigabe

Beispiel 2.4

int_stack → stack<int>

Eine Instanz vom Typ int_stack ist eine Folge von ganzen Zahlen (int). Eine Folge der Länge 0 heißt der leere Stack.

```
class int_stack{
   private: // Implementierung
   public:
      // Konstruktor
      stack(int sz); // Erzeugt einen Stack mit
            // maximaler Anfangsgröße sz.
      // Destruktor
      ~stack()
      void push(int x); // Fügt x als letztes
         // (Top-)Element an die Folge an.
      int top() const; // liefert das letzte
         //(Top-) Element
         // Precondition: Stack ist nicht leer.
      int pop(); // entfernt das letzte Element der
         // Folge und gibt es zurück.
         // Precondition: Stack ist nicht leer.
      bool empty() const; // liefert true zurück,
         // falls der Stack leer ist, sonst false
};
```

`const` ist ein Schlüsselwort. Steht es hinter einer Funktion, so gibt es an, dass dies eine Funktion ist die den Zustand des Objekts, für das sie aufgerufen wird, nicht ändert.

2.4 Trennung von Spezifikation (Definition) und Implementierung

Im Gegensatz zu Java existiert in C++ eine Trennung zwischen Spezifikation und Implementierung. Hier werden die Deklarationen einer Methode (ohne Rumpf) in einer speziellen Header-Datei (z.B. int_stack.h) angegeben. Die Implementierung (dh. der C++-Code der Methoden) wird in einer anderen Datei (z.B. int_stack.cpp) angegeben. In dieser wird über #include „int_stack.h“ die Header-Datei eingebunden.
In C++ ist es, im Gegensatz zu Java, möglich mehr als eine Klasse pro Datei zu implementieren.
Diese und weitere Unterschiede zwischen Java und C++ entstehen durch die unterschiedlichen Ziele der beiden Programmiersprachen. So ist beispielsweise Effizienz ein wichtiges Ziel von C++ im Gegensatz zur Sicherheit, die ein wichtiges Ziel in Java ist.

Beispiel 2.5

C++ (int_stack.h)

```
class int_stack{
   private int* A; // Feld
   int sz; // Länge von A
   int t; // Pointer (int*) auch möglich
}
```

C++ (int_stack.cpp)

```
#include "int_stack.h"
// Konstruktor
int_stack::int_stack(int n){
   sz = n;
   A = new int[sz];
   t = -1; // Stack leer
}
```

Java

```
private int[] A;
int t;
```

2.5 Integer-Stack: dynamisch

2.5.1 Zwei Implementierungen

Es gibt mehrere Möglichkeiten die Klasse int_stack zu implementieren.

I) Array-Datenstruktur

(int_stack.h)

```
class int_stack{
   public:
      // Konstruktor
      int_stack(); // Erzeugt leeren Stack
      // Destruktor
      ~int_stack();
   private:
      int* A;
      int sz; // Anfangsgröße. Kann später
            //vergrößert werden.
      int t;
      void push(int x);
}
```

Implementierung der Operationen

(int_stack.cpp)

```
int_stack::int_stack(){
   sz = 2; // Am Anfang Feld der Länge 2
   A = new int[sz];
   t = -1;
}
int_stack::~int_stack(){
   delete[] A; // Speicherfreigabe
}
void int_stack::push(int x){
   if(t == sz-1){ // Stack ist voll
      int* B = new int[2*sz];
      sz = 2*sz;
   }
   for(int i=0; i<=t; i++){
```

```
        B[i] = A[i]; // Werte kopieren
    }
    delete[] A;
    A = B;
    A[++t] = x; // Eigentlicher push
}
int int_stack::pop(){
    if(t == -1){
        EXCEPTION 'Pop auf leerem Stack';
    }
    if(t == sz/4){
        int* B = new int[sz/2];
        sz = sz/2;
    }
    for(int i=0; i<=t; i++){
        B[i] = A[i]; // Werte kopieren
    }
    delete[] A;
    A = B;
    }
    return A[t--];
}
```

Bemerkungen

- Der Destruktor wird automatisch aufgerufen, wenn eine Variable nicht mehr gültig ist.
- In Java löscht der Garbage Collector eine Variable automatisch, wenn er merkt, dass sie nicht mehr genutzt wird.
- Push-Operation: Verdopple das Feld, falls es zu klein wird.

Einschub: Variablen, Konstruktoren, Wertzuweisungen (C++ ↔ Java)

C++
int_stack S; // Konstruktor
Konstruiert Objekt (leeren Stack)



Java
S → Referenz/Pointer
↳ S = new int_stack();

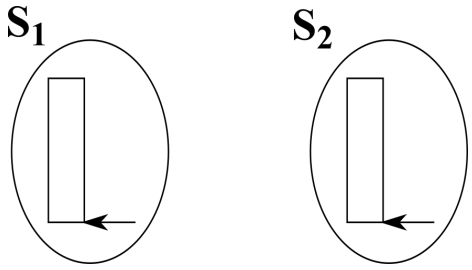



```
void func(){
    int_stack S; // lokal
    S.push(17);
    return S.pop();
    // Destruktor wird automatisch aufgerufen
}
```

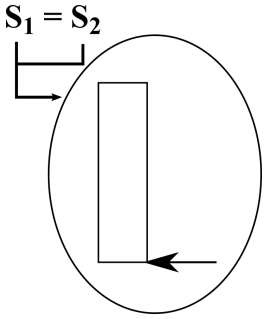
Value-Semantik (C++) ↔ Referenz-Semantik (Java)

Wertzuweisung

C++
$s_1 = s_2$;
Zwei Stacks mit gleichem Inhalt.
s_1 ist Kopie von s_2.



Java
int_stack s_1, s_2;
Die beiden Referenzen zeigen auf das gleiche Objekt.



Referenz-Semantik in C++

Um die Referenz-Semantik in C++ zu nutzen, müssen Pointer verwendet werden.

```
int_stack* sp1 = new int_stack();
int_stack sp2 = sp1;
```

Weitere Aspekte

- Test auf Gleichheit == Operator
- Parameterübergabe (und Rückgabe)

Beispiel 2.6

Parameterübergabe „by value“

```
int func(int_stack S){
   S.push(17);
   return S.top();
}
```

Java

```
int_stack S = new int_stack();
func(S);
```

S wird verändert!

C++

```
int_stack S;
func(S);
```

Die Übergabe wird nicht verändert, da sie „by value“ erfolgt d.h. der lokale Parameter S ist eine Kopie des aktuellen Parameters.

Parameterübergabe „by reference“
C++

```
int func(int_stack& S){
   S.push(17);
   return S.top();
}
```

S ist identisch mit dem aktuellen Parameter. Also wie in Java.

Pointer

```
int func(int_stack* S){
   (*S).push(17); // oder S -> push(17);
}
int_stack S;
func(&S);
int_stack * p = new int_stack();
func(p);
```

II) Einfach verkettete Liste

siehe Übung

2.5.2 Korrektheit einer Implementierung

(hier Array-Implementierung von int_stack)

Eigentlich hat man zwei Datentypen

1) Den abstrakten Datentyp int_stack (→ Datei stack.h)
2) Den konkreten Datentyp (Array-Implemtierung)

Abstrakter Zustand $s \in S$: Folge von int's
Konkreter Zustand $z \in Z$: Werte der (Member-)Variablen A, t, sz.
Durch die Invariante wird garantiert, dass nicht alle Kombinationen von A, t, sz möglich sind, sondern nur die gültigen konkreten Zustände:

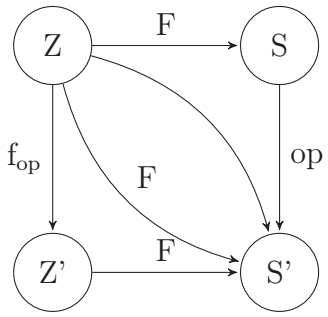
1) A ist ein Feld der Länge sz
2) $-1 \leq t \leq sz - 1$

Noch zu zeigen:

1) Konstruktoren erzeugen nur gültige konkrete Zustände
2) Für jede abstrakte Operation op (push, pop, ...) und die dazugehörige konkrete Operation f_{op} (Methode) zeige:

$$F(f_{op}(z)) = op(F(z))$$

Kommutatives Diagramm

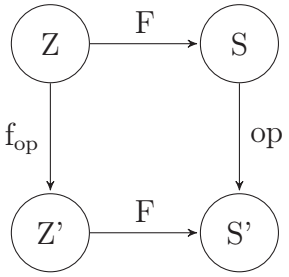


Funktion F (Semantik)

F: Z → S

hier:

$$F(A,t,sz) = \begin{cases} A[0], \ldots, A[t], & \text{für } t \geq 0 \\ \text{leer}, & \text{für } t = -1 \end{cases}$$



Beispiel 2.7 pop-Operation

$F(f_{op}(A, t, sz)) = pop(F(A, t, sz))$

Beweis (für jede Operation) → siehe Übung

3 Vererbung $\leftrightarrow$ Templates

Situation:

Es wird ein Datentyp A benötigt, der sehr ähnlich ist einem bereits definierten Datentyp B ($\rightarrow$ Klassen).

A soll

- einen Teil der Daten/Operationen wiederverwenden
- andere hinzufügen
- einige verändern (auf andere Wege implementieren)

Beispiel 3.1

Eine existierende Klasse: Polygon (B)
Eine neue Klasse: Rechteck (A)
Es handelt sich hierbei um eine Spezialisierung, da ein Rechteck ein spezielles Polygon ist.
Rechteck könnte alle Polygon-Operationen übernehmen (z.B. draw(), translate(dx,dy), rotate(x,y,ρ), area(), ...)

Einige Operationen können effizienter implementiert werden: Z.B. Flächenberechnung: area()
Andere Operationen sind nur für Rechtecke definiert: Z.B. Seitenverhältnis: ratio()

Allgemein: Klasse A (im Beispiel Rechteck) wird von Klasse B (im Beispiel Polygon) abgeleitet. B nennt man dabei die Basisklasse und A die abgeleitete Klasse.

Java

```
class A extends B{
   // Erweiterung: A-Teil
}
```

C++

```
class A: public B{
   // B-Teil:
   // Alle Daten und Methoden werden übernommen

   // A-Teil:
   // Zusätzliche Daten/Methoden,
   // modifizierte Operationen
}
```

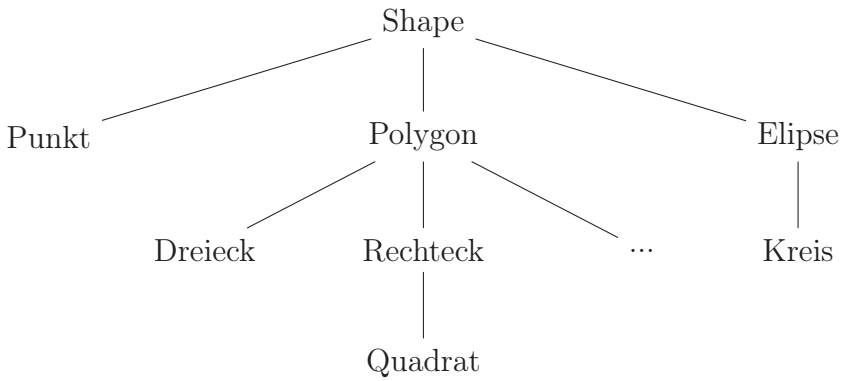
```
class Polygon{
   private // Daten

   double area();
   double umfang();
   void translate(double dx, double dy);
   void draw(...);
   ...
}
```

```
class Rechteck: public Polygon{
   // Konstruktoren -> Übung
   private:
      double b, h; // A-Teil
      // Überschreiben:
         double area(){return b*h;}
         double umfang(){return 2+(b+h);}
      // Hinzufügen:
      double ratio(){return b/h;}
}
```

Jedes Rechteck ist ein spezielles Polygon.

3.1 Allgemeiner: Ableitungsbaum



3.1.1 Typverträglichkeit

In C++ kann einer Variable von Typ B* (Pointer) oder B& (Referenz) ein Objekt (Pointer/Referenz) vom Typ A zugewiesen werden.
In Java sind alle Variablen vom Typ B.

C++

```
Polygon* p = new Rechteck(...);
void func(Polygon& poly){...}
Rechteck rect(...);
func(rect);
```

Java

```
Polygon p = new Rechteck(...);
```

Genauer: Alle im Ableitungsbaum verfügbaren Typen können zugewiesen werden.

Variable/Parameter hat zwei Typen:

- statischer Typ (zur Compile-Zeit bekannt (z.B. Polygon))
  → bestimmt, welche Methoden zur Verfügung stehen
- dynamischer Typ (zur Laufzeit bekannt (z.B. Rechteck))
  → bestimmt die Implementierung der Methoden

3.2 Polymorphe Datenstrukturen

Ein Beispiel wäre eine Feld von Polygonen.

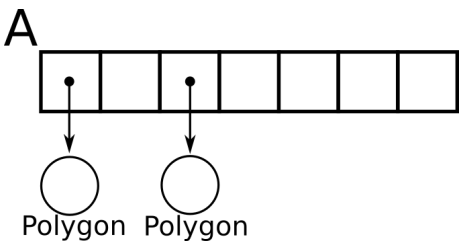
C++

```
Polygon** A = new Polygon*[100];
```

Java

```
Polygon[] A = new Polygon[100];
```

Polgon** A entspricht einem Feld von Pointern.




```
A[0] = new Polygon(...);
A[1] = new Rechteck(...);
A[2] = new Dreieck(...);
A[3] = new Quadrat(...);
```

→ polymorphes Array

Beispiel 3.2 Berechnung der Gesamtfläche

```
double total_area(Polygon** A){ // Java: (Polygon[] A)
   double total = 0;
   for (int i = 0; i < 100; i++){
      total += A[i]->area();
      // Oder: total += (*A[i]).area();
   }
   return total;
}
```

Der dynamische Typ bestimmt, welche Area-Implementierung verwendet wird.
→ Dynamische Bindung

Dynamische Bindung

```
polygon * p = new rechteck(...)
```

oder

```
double func(polygon& poly){
   return poly.area();
}
rechteck rect(...)
func(rect)
```

In C++ ist die statische Bindung der Standard. Für die dynamische Bindung wird eine virtuelle Methode benötigt.

```
virtual double area(){...}
```

`virtual` ist dabei das Schlüsselwort für die dynamische Bindung.

Beispiel 3.3 Grafik-Editor (erweiterbar)

Der Grafik-Editor speichert eine Menge von allgemeinen Flächen (→ shape)
Abstrakte Klasse „shape“

C++

```
class chape{
   virtual void draw(...) = 0
   virtual void area() = 0
   virtual void rotate(double phi, ...) = 0
}
```

Java

```
abstract class shape{
   abstract void draw(...);
   ...
}
```

Abstrakte Klassen werden verwendet, um Interfaces zu definieren.
Der Grafik-Editor speichert eine Liste von `shape*`.
`shape * p = new shape();` ergibt einen Fehler, da `shape` abstrakt ist.

Wenn `polygon` von `shape` abgeleitet ist:
C++

```
shape * p = new polygon(...)
```

```
class polygon: public shape{
   double area(){...}
}
```

Alle rein virtuellen (abstrakten) Methoden müssen definiert werden.

Java:

```
class polygon implements shape{...}
```

```
Interface shape{
   void draw();
}
```

Im Editor
`void draw_objects(){`
Iteriere über alle shapes s und rufe `s.draw()` auf.

Anwendung auf Algorithmen
Lineare Ordnung durch ein Interface (Java comparable)

C++

```
class comparable{
   virtual int compare (comparable& x) = 0;
```

Die Funktion vergleicht das Objekt, welches die Funktion aufruft, mit x und liefert

$$\begin{cases} +1, & \text{falls dieses } > x \\ 0, & \text{falls dieses } = x \\ -1, & \text{falls dieses } < x \end{cases}$$

Anwendung in generischen Sortieralgorithmen
Quicksort(comparable * A[])
zum Vergleich if(x.compare(y) ≤ 0) ...
Sortiere ein Feld von point:

```
class point : public comparable{
   point ...
   int compare(comparable & p){}
      // p ist tatsächlich ein Point!
   double px = ((point&) p).x;
   double py = ((point&) p).y;
   (x,y) < (px,py)
   // lexikographische Ordnung
   return ... ;
};
```

Aufruf von Quicksort

```
point* A [] = new point*[100];
(point**)
for( i=o; i<100; i++){
   A[i] = new point(i, i*i);
}
Quicksort(A, 100);
```

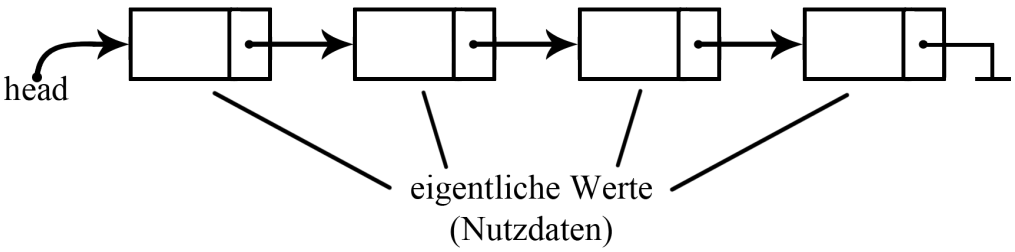
Datenstrukturen: → binäre Suchbäume, Schlüssel vom Typ „comparable“

3.3 Weitere Anwendungen von Vererbung

3.3.1 Generische Datenstrukturen

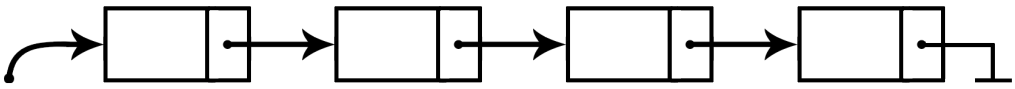
Eine generische Datenstruktur ist zum Beispiel eine Liste von beliebigen Objekten. Diese Liste soll leicht wiederverwendbar sein z.B. eine einfach verkettete Liste. Dadurch muss sie nicht immer neu implementiert werden.

Einfach verkettete Listen



Beobachtung:
Implementierung der Operationen (push, pop, ...) ist unabhängig vom Typ der Werte (int, string, point, ...).

Abstrahieren: Liste ohne Werte



1.) Basisklasse für allgemeine Listenelemente

```
class slist_elem{
   slist_elem* next;
   slist_elem(slist_elem* p){
      next = p;
   }
}
```

2.) Basisklasse für allgemeine Liste

```
class slist{
   slist_elem* first;
   slist(){
      first = NULL;
   }

   // hier eigentlich Destruktor

   void push(slist_elem * p){
      p->next = first
      first = p;
   }

   slist_elem* pop(){
      if(first == NULL) return first;
      slist_elem* p = first;
      first = first.next;
      return p;
   }
}
```

`slist` funktionier auch für alle von slist_elem abgeleiteten Klassen.

Beispiel 3.4 Liste von points

```
class point: public slist_elem{
   // wie vorher ...
}
point p
```

```
class point_list: public slist{
   // neues Interface, das nur Points erlaubt
   // -> Filter
   void push(point* p) {
      slist :: push(p);
   }
   point* pop(){
```

```
        return (point*) slist :: pop();
        // Casting ist sicher, da nur points in
        // der Liste sind
    }
    ...
}
```

1.) Leite point von slist_elem ab
2.) slist kann nun für point verwendet werden

```
slist L;
point* p new point(x,y);
L.push(p);
point* q = (point*)L.pop();
```

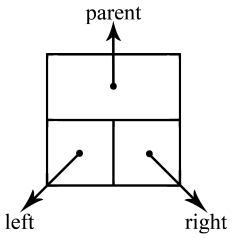
Man hat nun Polymorphie.
`L.push(new rechteck(...));` ist auch möglich.



→ doppelt-verkettete Listen

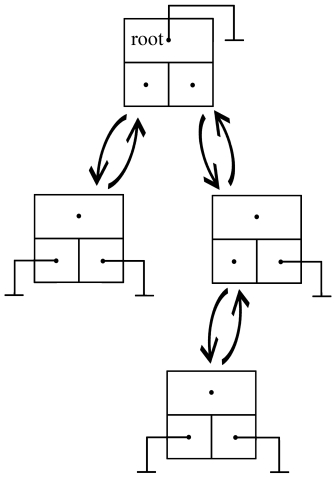
Aufwendigste Datenstruktur: z.B. balancierter Suchbaum z.B. AVL-Baum

Die 1. Klasse wird für die Knoten genutzt.




```
class bin_tree_node{
    bin_tree_node* left;
    bin_tree_node* right;
    bin_tree_node* parent;
};
```

Die 2. Klasse wird für den Baum genutzt.




```
class bin_tree{
   virtual int comp(bin_tree_node* p,
            bin_tree_node* q) = 0
   // Die Funktion comp(...) wird in einer
   // abgeleiteten Klasse definiert.
   void insert(bin_tree_node* p){
      // fügt p in den Baum ein und
      // verwendet comp zum Vergleich
   }
   bin_tree_node* lookup(bin_tree_node* p){
      if(comp(q,p) > 0)
         q = q->left;
      else
         q = q->right;
   }
};
```

Anwendung auf points

```
class point: public bin_tree_node{
   double x,y;
}
```

```
class point_bin_tree: public bin_tree{
   // Definiere comp-Funktion
   int comp(bin_tree_node* p, bin_tree_node* q){
      point* a = (point*) p;
      point* b = (point*) q;
      // lexikografische Ordnung:
      if(a->x < b->x) return -1;
      if(a->x > b->x) return +1;
      if(a->y < b->y) return -1;
      if(a->y > b->y) return +1;
      return 0;
   }
   void insert(point* p){
      bin_tree :: insert(p);
   }
   point* min(){
      return (point*) bin_tree :: min();
   }
```

bin_tree_node → point
bin_tree → point_bin_tree
Wiederverwendung von Cache!

Templates oder Schablonen

Eine weitere Anwendung von generische Datenstrukturen ist ein Template oder eine Schablone.

1) Funktionstemplates
Ein Funktionstemplate ist eine „Schablone“ die angibt, wie eine Funktion erzeugt werden soll. Aus einem solchen Template können Funktionen mit gleichem Ablauf aber verschiedenen Parametertypen erzeugt werden. Die Funktion hat also immer die gleiche Parameteranzahl aber die Parameter können unterschiedliche Datentypen haben.

```
template<class T> // "class" kann durch
   // "typename" ersetzt werden
   void swap(T& x, T& y){
      T tmp = x;
      x = y;
      y = tmp;
   }
```

Wichtig: Die Implementierung ist nicht abhängig vom Typ T.
Anwendung:

```
double a = 3.14;
double b = 15;
swap(a,b);
```

Der Compiler instanziiert das Template für T = double.
Ähnliche Funktionen sind Minimum (min(x,y)) und Maximum (max(x,y))

2) Klassentemplates
Klassentemplates lassen sich im Gegensatz zu Funktionstemplates nicht überladen.

Beispiel für einen Stack:

```
template<class T>
   class stack{
      T* A; // Feld von T's
      int sz;
      int t;
      public:
         void push(T x){...}
         T pop(){...}
   }
```

Verwendung:

```
stack<int> S;
stack<point> S;
```

Beispiel 3.5 Beispiel für mehrere Typen

```
template<class K, class I>
   class dictionary{
      // Wörterbuch mit Schlüssel vom Typ K
      // und Informationen vom Typ I
      // mögliche Implementierung: AVL-Baum
      void insert(K, I){
         // trägt neues Paar ein
      }
      bool lookup(K key){
         // testet, ob Eintrag mit Schlüssel key
            vorhanden ist.
      }
      I translate(K key){...}
      void remove(K key){...}
   }
```

Anwendung: Word-Count
Zähle wie oft einzelne Wörter in einem Text vorkommen.
`dictionary<string, int> D;` speichert Paare (S, i). Der String S ist bis jetzt i mal vorgekommen.

Templates in Kombination mit Vererbung am Beispiel einer einfach verketteten Liste:

```
template<class T>
   class list :: public slist{
      void push(T* x){slist :: push(x);}
      T pop(){return (T*) slist :: pop();}
      ...
   }
```

analog: bin_tree<T>

4 Fortgeschrittene Datenstrukturen und Algorithmen

LEDA: Library of Efficient Data Types and Algorithms

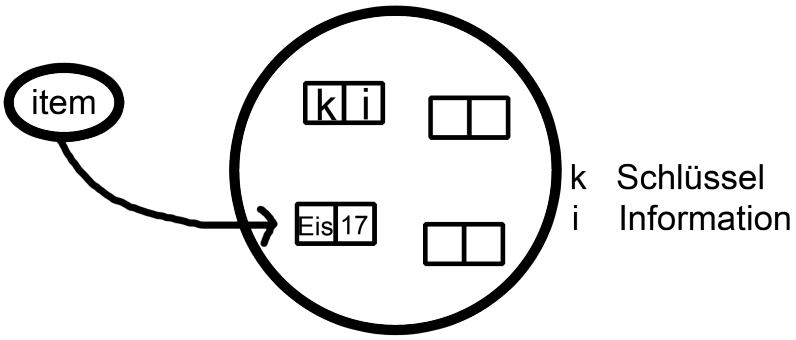
4.1 Spezifikation von Datentypen in LEDA

4.1.1 Item-Konzept

Viele Datenstrukturen werden dabei definiert als eine Menge von sogenannten Items.

Beispiel 4.1 Dictionary
`dictionary<string, int> D` speichert Paare aus Schlüseln (strings) und Informationen (int).

D ist eine Menge von Items (dic_item).



Operationen

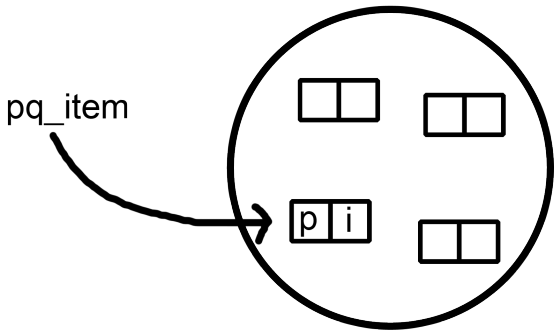
```
dic_item D.insert(string s, int i)\\
   if( D enthält kein Item mit Schlüssel s){
      Füge Item (s,i) ein;
      return (s,i);
   }
   else{
      Ersetze die Information durch i;
      return (s,i);
   }
}
dic_item D.lookup(string s){
   Liefert das Item mit Schlüssel s
   if(kein solches existiert)
      return NULL;
}
void D.change_inf(dic_item it, int x){
   ersetze die Information von it durch x
}
void delete_item(dic_item it){
   Item: Zugriff über Position (Abstraktion von den
      Begriffen Pointer, Referenz, Index)
}
```

Vergleich verschiedener Spezifikationen
Dictionary als Funktion $D : K \to I$ $\leftrightarrow$ Item-Konzept

Beispiel 4.2 Priority Queues
Da bei Priority Queues immer ein Extremwert gesucht wird, eigenen sich für die Implementation Fibonacci-Heaps besonders gut, da eine Abfrage des Extremwertes in $\mathcal{O}(1)$ möglich ist.

`priority_queue<P,I> PQ;`
P: Priorität z.B. Zahlen (int)
I: Information z.B. Knoten eines Graphen (node)
`priority_queue<int, node> PQ;`

PQ ist eine Menge von Items (pq_item)



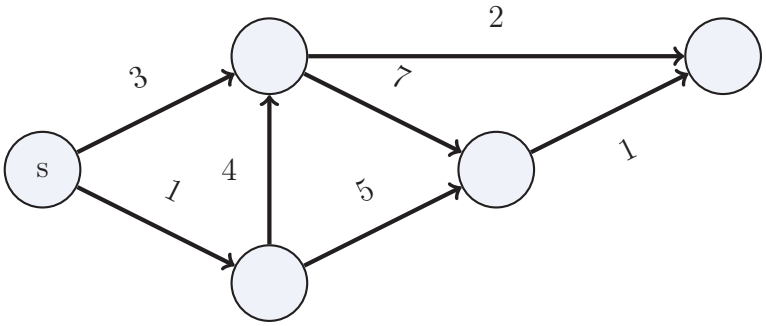
Operationen

```
pq_item PQ.insert(P prio, I inf)\\
      Fügt Item (prio, inf) ein; // Im Gegensatz zum
         Dictionary dürfen bei der Priority-Queue
         Wiederholungen auftreten.
      return (prio, inf);
}
PQ.findmin(){
   Liefert ein Item mit minimaler Priorität.
   Falls die Queue leer ist, wird NULL zurück gegeben.
}
P PQ.prio(pq_item it)
I PQ.inf(pq_item it)
void PQ.delmin(){
   Löscht ein minimales Item.
}
void PQ.decrease_p(pq_item it, P q){
   Vermindert die Priorität von it auf q.
   (q ist kleiner gleich der alten Priorität von it)
}
```

Anwendung auf kürzeste Wege Algorithmen

Speziell: Dijkstra's Algorithmus

Beispiel 4.3



Eingabe

- Graph $G = (V, E)$
- Kostenfunktion $cost : E \to int^*$
- Startknoten $s \in V$

Ausgabe

- Distanzfunktion $dist : V \to int^*$
  $dist(v)$ = Kosten eines billigsten Pfades von s nach v.
  Kosten eines Pfades ist die Summe des billigsten Pfades.

Idee von Dijkstra

1. Überschätze die Dist-Funktion

$$dist(v) := \begin{cases} \infty, & \text{falls } v \neq s \\ 0, & \text{falls } v = s \end{cases}$$

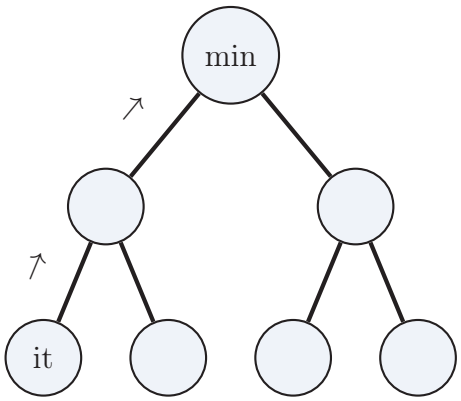
2. Kandidatenliste $\mathcal{U}$ = Menge aller Knoten aus denen Kanten ausgehen können, die eine Abkürzung darstellen. ($\to$ Verletzung Dreiecks-Ungleichung)
   Am Anfang ist nur der Startknoten s in der Liste, da $dist(s) = 0$

3. Hauptschleife über $\mathcal{U}$
   Idee von Dijkstra:
   i) Wähle jeweils $u \in \mathcal{U}$ mit $dist(u)$ minimal. Entferne den gewählten Knoten u aus der Menge $\mathcal{U}$.
   ii) Beobachtung: $dist(u)$ ist korrekt.
   iii) Durchlaufe alle aus u ausgehenden Kanten und überprüfe Dreiecks-Ungleichung.

```
forall_out_edges(e,u){
    v = target(e)
    d = dist(u) + cost(e)
    if(d < dist(v)) {
        dist(v) = d
        U = U + v
    }
}
```

`PQ.decrease_p(it, q)` kann effizient realisiert werden.
Fibonacci-Heap $\rightarrow \mathcal{O}(1)$ (amortisiert)

Binärer Heap:



Iteration:
In C++ eignen sich dafür Makros. Makros sind kleine Programmstücke, die über ihren Namen an beliebigen Stellen im Code aufgerufen werden können. Wie bei Funktionen auch, können ihnen Parameter übergeben werden. Datentypen werden dabei ignoriert, da die Variablen einfach nur ersetzt werden durch die übergebenen Werte.

```
# define
   forall_items(it,D) // D ist die Datenstruktur
      for(it = D.first_item()){
         if(it != NULL)
            it = D.next_item(it)
      }
```

Eine mögliche Anwendung wäre das Dictionary, bei dem alle Schlüssel (dic_it) ausgegeben werden sollen.
In der Graphtheorie sind Makros zum Beispiel interessant, da sehr oft über Adjazenzen, Kanten oder ähnliches iteriert wird.

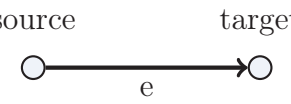
4.2 Graphen und Graphalgorithmen (in LEDA)

Der allgemeine Graph-Datentyp in LEDA: „*graph*“
Der Datentyp repräsentiert gerichtete Graphen.
Ein Graph G besteht aus zwei Typen von Objekten ($\rightarrow$ Items):

Knoten vom Typ „*node*“

Kanten vom Typ „*edge*“

Mit jedem Knoten v sind zwei Listen vom Typ $list < edge >$ assoziiert:
out_edges und in_edges.
Eine Kante verläuft zwischen genau 2 Knoten.



4.2.1 Operationen auf einem Graphen G

- Access

  $node\ G.source(edge\ e);$
  $node\ G.target(edge\ e);$
  $int\ G.outdeg(node\ v);$
  $int\ G.indeg(node\ v);$
  $list < edge >\ G.out_edges(node\ v);$
  ...

- Update

  $node\ G.new_node()$;
  Erzeugt einen neuen Knoten in G und gibt ihn (seine Referenz) zurück.
  $edge\ G.new_edge(node\ v, node\ w)$
  $void\ G.del_edge(edge\ e)$
  $void\ G.del_node(nodev)$
  Entfernt v und alle inzidenten Kanten.
  ...

- Iteration (Makros)

  $forall_nodes(v, G)\{\dots\}$
  $forall_edges(e, G)\{\dots\}$
  $forall_out_edges(e, v)$
  $forall_in_edges(e, v)$

Iteration über Nachbarknoten von v

```
forall_out_edges(e,v){
   node w = G.target(e);
   // tue etwas mit w
}
```

Test, ob G azyklisch

Ein azyklischer Graph enthält keine Kreise.

Idee: Topologische Sortierung
Entferne jeweils einen Knoten v mit $indeg(v) = \emptyset$ bis der Graph G leer ist.
Falls wir keinen solchen Knoten finden, ist G zyklisch.
Falls G am Ende leer ist, ist G azyklisch.

```
ZERO ← {v ∈ V: indeg(v) = 0}
   while(ZERO ≠ ∅) {
      u ← bel. Knoten aus ZERO
      ZERO ← ZERO \ {w}
      forall(v ∈ V mit (u,v) ∈ E){
         entferne (u,v) aus G
         if(indeg(v) = 0){
            ZERO ← ZERO ∪ {v}
         }
      }
      entferne u aus G
   }
```

(Realisiere die Menge ZERO durch einen Stack.)

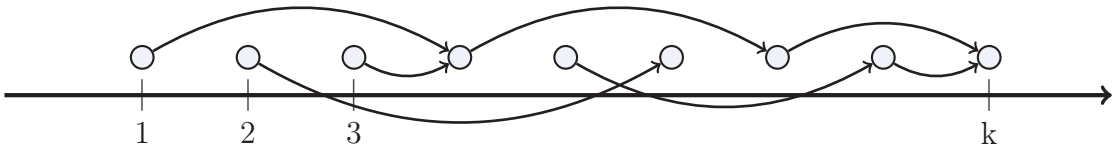
```
bool isAcyclic(graph G){ // Übergabe by value (Kopie)
   stack<node> ZERO;
   node v;
   forall_nodes(v,G){
      if(G.indeg(v) == 0)
         ZERO.push(v);
   }
   while(!ZERO.empty()){
      node u = ZERO.pop();
      edge e;
      forall_out_edges(e,u){
         node w = G.target(e);
         G.del_edge(e);
         if(G.indeg(w) == 0)
            ZERO.push(w);
      }
      G.del_node(u);
   }
   return G.empty();
}
```

Erklärung:
Zuerst werden alle Knoten, die keine eingehenden Kanten besitzen in die Menge *ZERO* aufgenommen. Danach wird solange noch Knoten in der Menge *ZERO* ent-

halten sind, jeweils einer entfernt und für diesen Knoten v überprüft, ob einer der Knoten w, die direkt über eine von v ausgehende Kante mit v verbunden sind, keine eingehenden Kanten mehr hat, wenn diese eine Kante gelöscht würde. Ist dies der Fall, wird w in die Menge $ZERO$ aufgenommen.

Varianten

1. Topologisches Sortieren
   Abb. $ord : V \to \{1, \ldots, n\}$
   (d.h. Anordnung der Knoten)
   1) ord ist injektiv
   2) $\forall (v, w) \in E : ord(v) < ord(w)$



2. G soll nicht zerstört werden
   d.h. Löschen der Kanten wird nur simuliert.
   Idee:
   i) Speichere die Indeg-Werte in einem Feld `indeg[v]`
   ii) Statt `G.del_edge(e)` → `indeg[G.target(e)]-`
   Dadurch können Referenz-Parameter (`graph& G`) genutzt werden und es wird Speicherplatz gespart, da keine zusätzliche Kopie angelegt werden muss.

4.2.2 Basis-Graphtyp graph

Die Informationen (Daten) werden mit den Knoten und Kanten gespeichert.

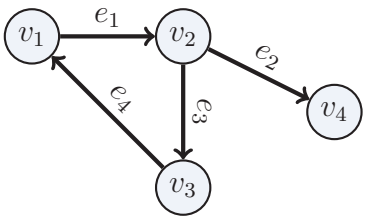
1) Parametrisierte Graphen
   `GRAPH<vtype, etype> G;`
   → Netzwerk: Knoten stehen für Objekte vom Typ `vtype`, Kanten vom Typ `etype`.

   Beispiele:
   ```
   GRAPH<stadt, autobahn> Map;
   GRAPH<transistoren, wire> Schaltkreis;
   GRAPH<person, beziehung> Gruppe;
   ```

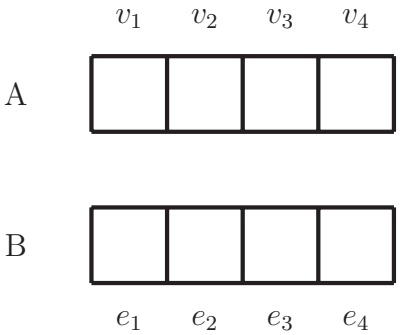
Ableitung: graph → `GRAPH<vtype, etype> G;`

Verwendung der allgemeinen Algorithmen (für den Basistyp)
`bool is_acyclic(graph G)`
`bool is_acyclic(Map)`
Wegen der Typverträglichkeit ist dies möglich.

2) Node/EdgeArray (LEDA)
`node_array<T> A(G)`
`edge_array<T> B(G)`



Getrennt vom Graphen werden die Felder gespeichert.



$x \leftarrow A[v]$
$y \leftarrow B[e]$
`A.operator[]node(v)`

Kommt ein Knoten hinzu können die Felder A und B nicht mehr genutzt werden, da ihre Größe statisch ist. Um das zu umgehen, hat jeder Knoten und jede Kante einen eindeutigen Index. `vec[v → index]`

```
void DIJKSTRA(graph& G, edge_array<int>& cost,
          node s, node_array<int>& dist){...}
```

Eingabe: `graph& G, edge_array<int>& cost, node s`
Ausgabe: `node_array<int>& dist`
Ein- und Ausgabedaten werden getrennt vom Graphen übergeben → generisch

Beispiele:

```
GRAPH<stadt, autobahn> Map,
edge_array<float> km(Map),
edge e,
forall_edges(e,Map) km[e] = Map.get_info(e).km;
node_array<float> dist(Map)
DIJKSTRA(Map,km,trier,dist);
```

3) temporäre Daten
`bool TOPSORT(graph& G, node_array<int>& out)` berechnet eiine topologische Sortierung von G d.h. $forall(v, w) \in Eord[v] < ord[w]$
Der Rückgabewert ist „true“ falls G azyklisch ist, ansonsten „false“

```
// temporäres INDEG-Feld simuliert das Löschen von
   Knoten
node_array<int> INDEG(G);
node v;
list<node> ZERO;
forall_nodes(v,G){
   INDEG[v] = G.indeg(v);
   if(INDEG[v] == 0)
      ZERO.append(v);
}
```

Anwendung DIJKSTRA

```
void Dijkstra(const graph& G, node s, const edge_array<
   int>& cost, node_array<int>& DIST){
   p_queue<node,int> PQ;
   node_array<pq_item> I(G,NULL);
   I[s] = PQ.insert(s,0); // I ist Menge der Indexe der
                          //Knoten
   node v;
   forall_nodes(v,G){
      DIST[v] = MAXINT;
   }
   DIST[s] = 0;

   while(!PQ.empty()){
      node u = PQ.del_min();
      egde e;
      forall_out_edges(e,u){
         node v = G.target(e);
         int d = DIST[u] + cost[e];
         if(d < DIST[v]){ // Dreiecks-Ungleichung verletzt
            if(DIST[v] == MAXINT)
               I[v] = PQ.insert(v,d); // zum ersten Mal
                                      //besucht
            else // v ist in PQ
               PQ.decrease_p(I[v],d)
            DIST[v] = d;
         }
      }
   }
}
```

Erklärung:
Zuerst wird s in die Priority Queue PQ aufgenommen. Die *while*-Schleife läuft nun so lange, bis PQ leer ist. Ist dies der Fall, ist zu jedem Knoten ein kürzester Weg bekannt, falls der Graph azyklisch ist. In der *while*-Schleife wird zuerst aus der PQ ein Knoten u gewählt. Dabei wird darauf geachtet, dass es eine perfekte Wahl ist, d.h. das der Knoten mit dem kleinsten, in der Priority Queue vorkommenden, DIST-Wert gewählt wird. Dann werden alle von u ausgehenden Kanten betrachtet, die jeweilige Δ-Ungleichung überprüft und falls sie verletzt wurde, der erreichte Knoten

entweder in PQ aufgenommen, falls er noch nicht in PQ drin ist oder sonst seine Priorität auf seinen neuen DIST-Wert gesetzt. Danach wird noch sein $DIST$-Wert angepasst.

Analyse
$\left.\begin{array}{l} n \times delmin \\ n \times insert \end{array}\right\} n \times log_2(n)$
$m \times decrease_p$ ($\mathcal{O}(1)$ amortisiert) $\}\mathcal{O}(m)$

Gesamtlaufzeit: $\mathcal{O}(m + n * log(n))$

Weiterer Aspekt: Korrektheit der Implementierung

Idee: Füge Programmcode hinzu, der für die Konkrete Eingabe testet, ob das Ergebnis korrekt ist.
→ (Programm Checker) Verifying Algorithms

hier: Teste für jede Kante am Ende die Δ-Ungleichung.

```
edge e;
forall_edges(e,G){
   node v = G.source(e);
   node w = G.target(e);
   ASSERT(DIST[v] + cost[e] >= DIST[w])
}
```

5 Netzwerkflussprobleme

Ein Unternehmer möchte täglich möglichst viele seiner Waren von einem seiner Standorte zu einem anderen Standort transportieren. Er nutzt dazu LKW‘s anderer Unternehmen, die zwischen verschiedenen Städten hin und her fahren. Jeder hat eine maximale Kapazität, die er von einer zu einer andern Stadt transportieren kann. Jeder dieser LKW‘s hat allerdings eine maximale Kapazität an Waren, die er transportieren kann. Da es in den einzelnen Städten keine Lagerhäuser gibt, können die Waren nicht zwischengelagert werden, sondern müssen sofort weiter transportiert werden.
Der Ausgangsstandort in diesem Szenario stellt den Startknoten s und der zweite Standort den Endknoten t dar. Die LKW ‘s fahren entlang der Kanten und ihre Kapazität entspricht der Kapazität der jeweiligen Kante.

Ziele

i) MaxFlow-Problem
Maximiere den Transport von einem Knoten s (source, Quelle) zu einem Knoten t (target, Senke)
Beispiel: Transport von Tonnen Stahl pro Tag über das Eisenbahnnetz.

ii) MinCostFlow
Konkretes Transportproblem (z.B. zwischen Produzenten und Konsumenten)
Ziel: Minimiere die Transportkosten.
Dazu wird zusätzlich du der Kapazität jeder Kante auch Kosten zugeordnet.

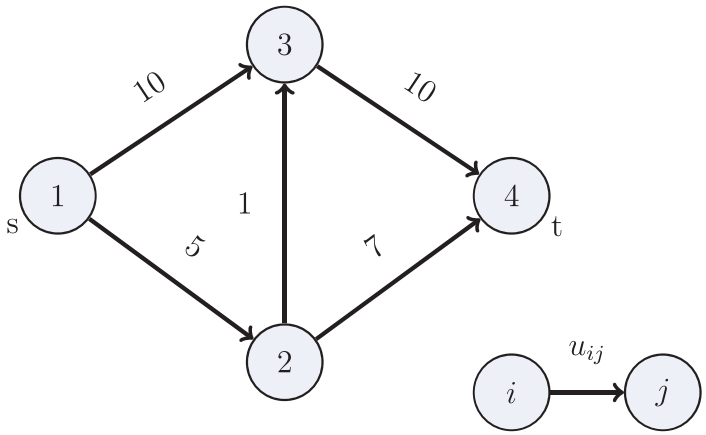
5.1 MaxFlow-Problem

5.1.1 Formale Definition

Gegeben sei

- ein gerichteter Graph $G = (V, E)$
- eine Kapazitätsfunktion $u : E \to \mathbb{R}_0^+$ und
- zwei Knoten $s, t \in V$ mit $s \neq t$.

$u((v,w))$ heißt Kapazität von (v,w). s heißt Quelle (source) und t Senke (target).



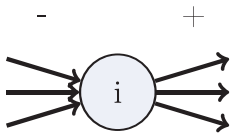
Gesucht ist eine Flussfunktion $x : E \to \mathbb{R}_0^+$ mit folgenden Eigenschaften:
(x_{ij} ist der Fluss über die Kante (i,j)) mit:

i) Kapazitätsbedingung:
$\forall\ e \in E : 0 \leq x(e) \leq u(e)$

ii) Massenbalance-Bedingung:
Sei $v \in V$ (beliebig aber fest!)

$$\underbrace{\sum_{(v,u)\in E}}_{\substack{\text{gehe über alle}\\ \textbf{ausgehenden}\\ \text{Kanten von v}}} x((v,u)) - \underbrace{\sum_{(w,v)\in E}}_{\substack{\text{gehe über alle}\\ \textbf{eingehenden}\\ \text{Kanten von v}}} x((w,v)) = \begin{cases} F, & v = s \\ 0, & \forall\ v \notin \{s,t\} \\ -F, & v = t \end{cases}$$

$F \geq 0$



$$\delta(i) = \sum x((i,j)) - \sum x((k,i))$$

ii) Optimalitätsbedingung:
F soll maximal sein.

Erklärungen:

i) Für alle Kanten muss gelten, dass ihr Flusswert zwischen Null und ihrer maximalen Kapazität liegt.

ii) Für alle Knoten die nicht s oder t sind, muss gelten, dass alles was in sie rein fließt auch wieder abfließt. Es kann also nichts in diesen Knoten gelagert werden. F entspricht dem Fluss der fließt. Aus dem Knoten s fließt der komplette Fluss F heraus, aber nichts herein. Beim Knoten t ist es genau umgekehrt.

iii) Ohne diese Bedingung wäre das Problem trivial, da der Null-Fluss ($x = 0$) die anderen beiden Bedingungen immer erfüllt.

Notation:
Knoten: i, j
Die Kapazität wird anstatt mit $u((i, j))$ jetzt nur noch mit u_{ij} und die Flussfunktion anstatt mit $x((i, j))$ mit x_{ij} bezeichnet.

Beobachtung

$\delta(t) = -\delta(s)$

1. Idee für einen Algorithmus: Erhöhe Pfade

i) Starte mit $x = 0(\forall\ (i, j) \in E\ x_{ij} \leftarrow 0)$

ii) Erhöhe x entlang von Pfaden von s nach t

5.1.2 Das Restnetzwerk

(residual network)
Ein Restnetzwerk beschreibt mögliche Flussrichtungen.

Definition

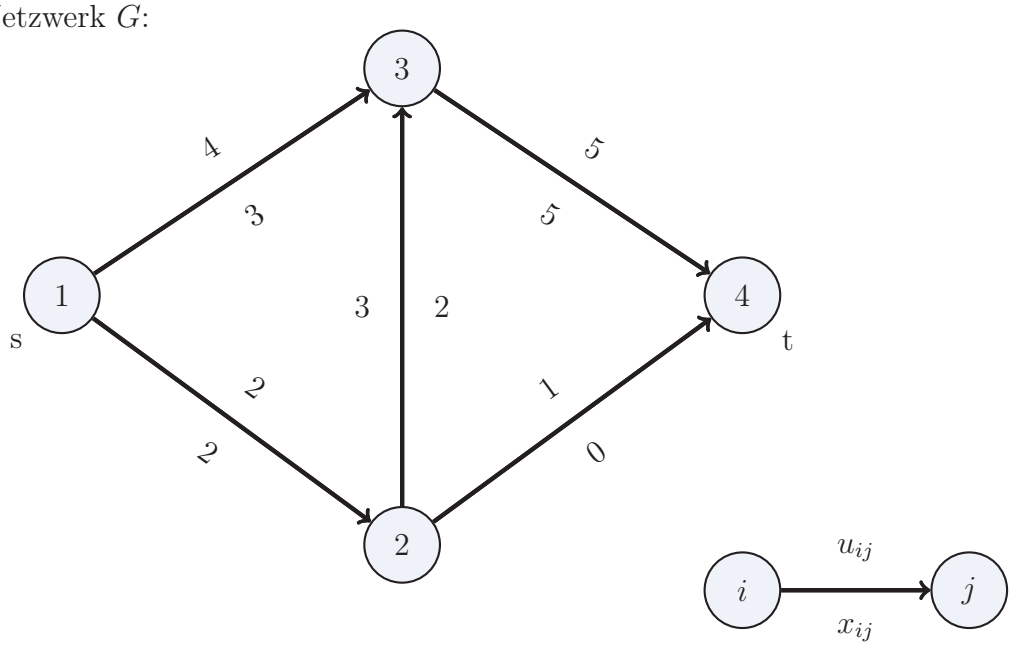
Sei x eine aktuelle Flussfunktion, dh. sie erfüllt die Kapazitätsbedingung (z.B. der Nullfluss ($\forall\ i, j\ x_{ij} = 0$)).
Das Restnetzwerk $G(x)$ besteht aus allen Kanten von G und deren Gegenkante. Es existieren also für jede Kante (i, j) aus G zwei Kanten in $G(x)$, nämlich die Kante (i, j) und ihre Gegenkante (j, i). Sie besitzen die Restkapazitäten $r_{ij} = u_{ij} - x_{ij}$ und $r_{ji} = x_{ij}$.



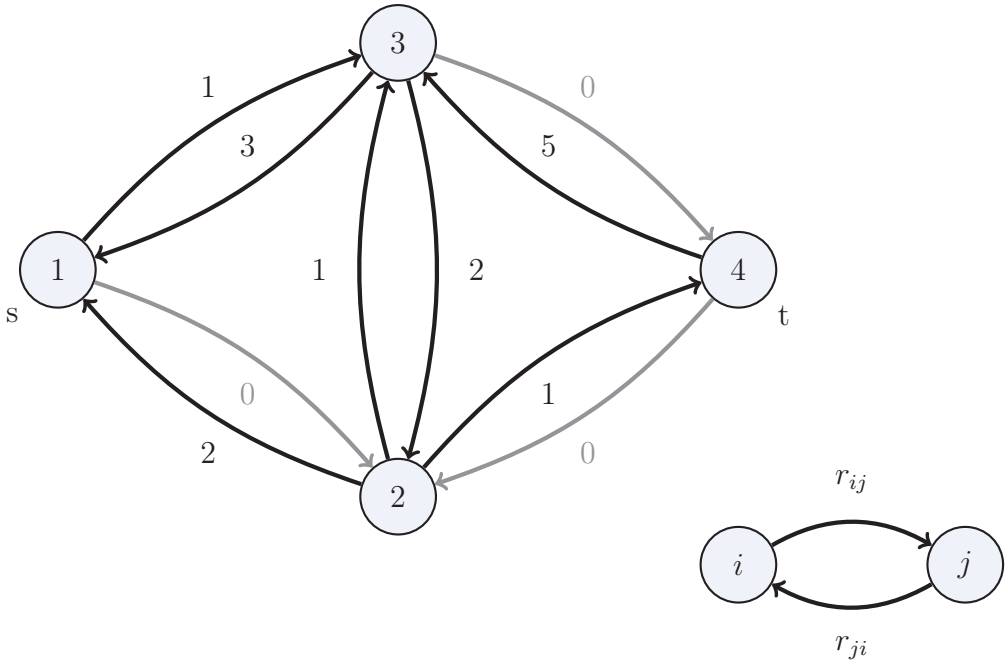
Die Restkapazitäten beschreiben mögliche Änderungen an der Flussfunktion. Eine Erhöhung geschieht in Richtung der Kante (i, j) (max $r_{ij} = u_{ij} - x_{ij}$) und eine Verminderung in die Gegenrichtung (max $r_{ji} = x_{ij}$).

Beispiel 5.1

Netzwerk G:



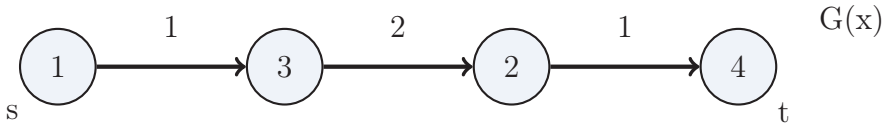
Aktueller Flusswert: $F = 5$
Restnetzwerk $G(x)$:



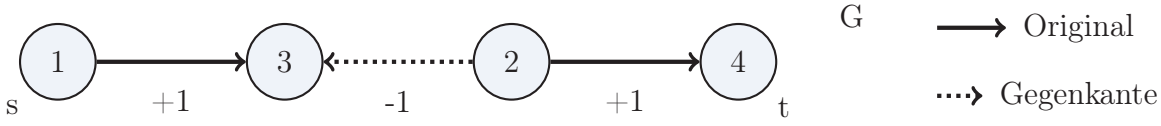
Da über Kanten mit einer Restkapazität von Null, nichts mehr fließen kann, werden sie weggelassen. Dadurch ist die folgende Beobachtung möglich.

Beobachtung: (ohne 0-Kanten)
Wenn alle Null-Kanten im Graphen weggelassen werden, entspricht die Frage ob ein Pfad existiert der Frage ob ein erhöhender Pfad existiert.
Jeder Pfad in $G(x)$ von s nach t beschreibt eine mögliche Erhöhung des Gesamtflusses F. ($\rightarrow$ Pfaderhöhung)

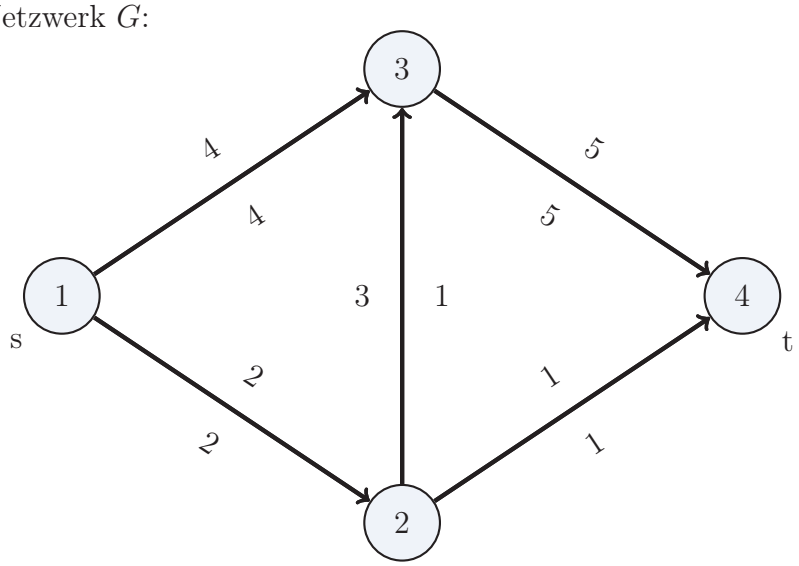
Im Beispiel 5.1 existiert nur ein solcher Pfad:



Eine Erhöhung um $\delta = min\{r_{ij}|(i,j) \in P\} = 1$ ist möglich.

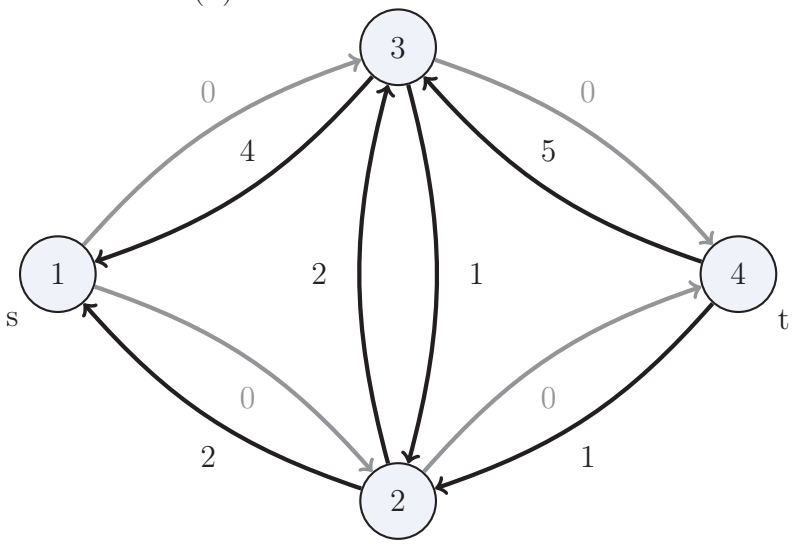


Netzwerk G:



Flusswert $F = 6$

Restnetzwerk $G(x)$:



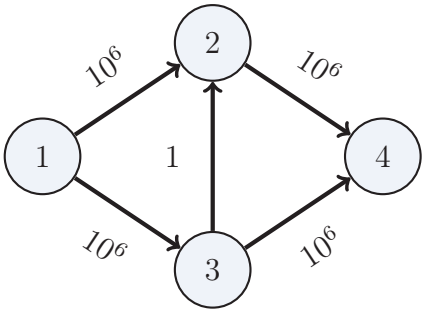
Hier existiert in $G(x)$ kein Pfad mehr von s nach t. Daher ist keine weitere Erhöhung mehr möglich.

Anmerkungen:

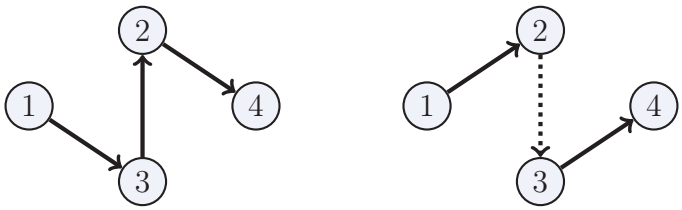
- Für den Nullfluss ($x = 0$) gilt $G(x) = G$.
- Wird eine Kante (i, j) in G saturiert (dh. $x_{ij} = u_{ij}$), so wird die Kante (i, j) aus $G(x)$ entfernt.
- Setzt man $x_{ij} = 0$, so wird die Gegenkante (j, i) aus $G(x)$ entfernt.

$\Rightarrow$ Änderungen an der Flussfunktion führen zu Änderungen in $G(x)$.

Bemerkung
Der einfache Pfaderhöhungsalgorithmus kann sich sehr schlecht verhalten.



Im schlechtesten Fall



2×10^6 Pfaderhöhungen

Korrektheit

i) x ist immer ein Fluss (Kapazitätsbedingung + Massenbalance-Bedingung)

ii) terminiert mit Flus x, sodass in $G(x)$ kein Pfad von s nach t existiert.

Beweis
Beweis mit Hilfe vom MaxFlow/MinCut-Theorem.

Implementierung

1. Keine expliziete Darstellung von G(x).
   Stattdessen: Navigiere im Originalgraphen.
   Verwende ausgehende Kanten, wenn $u_{ij} > x_{ij}$ und
   eingehende Kanten, wenn $x_{ij} > 0$.
2. Berechnung eines erhöhenden Pfades
   Verwende DFS oder BFS vom Knoten s aus und berechne die Kanten wie in 1. beschrieben. Stoppe, wenn t erreicht wurde.
3. Darstellung des Pfades durch pred-Verweise (Vaterverweise im DFS- oder BFS-Baum).

5.1.3 Pfaderhöhung: (Maxflow) s, t

Idee: Versuche im Restnetzwerk einen Pfad mit Restkapazität zu finden, indem z.B mit `explorefrom(s)`, `dfs(s)` oder `bfs(s)` alle von s aus erreichbaren Knoten gesucht werden, bis t gefunden wird.

Labeling-Algorithmus

```
void MF_Labeling(const graph& G, node s, node t,const
   edge_array<int>& cap, edge_array<int>& flow){
   node_array<bool> labeled; // besuchte Knoten
   node_array<edge> PRED; // Pfade

   // Initialisierung
   edge e;
   forall_edges(e,G){
      flow[e] = 0
   }
   node v;
   forall_nodes(v,G){
      labeled[v] = false
      pred[v] = NULL
   }

   while(true){
      queue<node> L; // Breitensuche
      labeled[s] = true;
      L.append(s);
      while(!L.empty()){
         node v = L.pop();
         // Iteriere über alle im Restnetzwerk adjazenten Kanten
         edge e;
         forall_out_edges(e,v){
            if(flow[e] == cap[e]) // Restkapazität ist null
               continue; // e nicht in G(x)
            node w = G.target(e);
            if(labeled[w])
               continue;
            labeled[w] = true;
            PRED[w] = e;
```

```
            L.append(w);
            }
            forall_in_edges(e,v){
            if(flow[e] == 0) continue;
            node w = G.source(e);
            if(labeled[w]) continue;
            labeled[w] = true;
            PRED[w] = e;
            L.append(w);
         }
         if(labeled[t])
            L.clear();
      } // Ende while-Schleife
      if(labeled[t])
         AUGMENT(G,s,t,PRED,cap,flow);
      else
         break; // ex. kein erhöhender Pfad
   }
}
```

Listing 5.1: MF_Labeling

```
void AUGMENT(const graph& G, node s, node t, const
   node_array<edge>& PRED, const edge_array<int>& cap,
   edge_array<int>& flow){
   int delta = MAXINT; // Restkapazität von P
   node v = t;
   while(v != s){
      int r;
      edge e = PRED[v];
      if(v == G.source(e)){ //Rückwärtskante
         r = flow[e];
         v = G.target(e);
      }
      else{ // Vorwärtskante
         r = cap[e] - flow[e];
         v = G.source(e);
      }
      if(r < delta)
         delta = r; // min
   }
```

```
    // Eigentliche Flusserhöhung
    v = t;
    while(v != s){
        edge e = PRED[v];
        if(v == G.source(e)){ //Rückwärtskante
            flow[e] = flow[e] - delta;
            v = G.target(e);
        }
        else{ // Vorwärtskante
            flow[e] = flow[e] + delta;
            v = G.source(e);
        }
    }
}
```

Listing 5.2: Augment

Erklärung:
Die äußere *while*-Schleife läuft so lange, bis explizit aus ihr herausgesprungen wird. Dies geschieht, wenn nach der inneren Schleife t nicht gelabelt ist, es also keinen erhöhenden Pfad mehr gibt.
Da s der Startknoten ist und jeder von s aus erreichte Knoten gelabelt und zur Menge L hinzugefügt wird, wird s zu Beginn immer gelabelt und in L eingefügt. In der inneren Schleife wird zuerst ein beliebiger Knoten u aus der Menge L entnommen. Von diesem Knoten u aus werden nun sowohl die ausgehenden, als auch die eingehenden Kanten betrachtet. Dabei werden diejenigen Kanten, die keine Restkapazität mehr besitzen, sofort aussortiert und nicht weiter betrachtet. Bei den restlichen wird überprüft, ob der durch die Kante erreichbare Knoten schon gelabelt wurde. Ist dies nicht der Fall wird er gelabelt, sein $PRED$-Verweis auf die Kante gesetzt, über die er erreicht wurde und der Knoten in die Menge L aufgenommen.
Die innere Schleife bricht ab, sobald L leer ist, was der Fall ist, wenn t gelabelt wurde, da dann ein erhöhender Pfad existiert und entlang diesem, im Algorithmus AUGMENT, Fluss geschickt wird.
Hierbei wird zuerst über den gefundenen Pfad von t nach s gelaufen, und dabei die maximale Flusserhöhung ausfindig gemacht. Ist diese gefunden, also s erreicht, wird erneut über den gefundenen Pfad von t nach s gelaufen und dabei die Flussänderung um den herausgefundenen Wert durchgeführt.
Es wird immer von t nach s gelaufen, da man nur den Vorgänger, nicht aber den Nachfolger eines Knotens kennt.

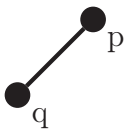
6 Algorithmische Geometrie

(Computational Geometry)

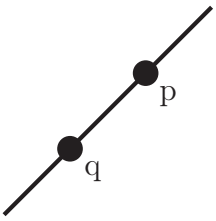
Die Objekte (Klassen / Typen) liegen hier in der Ebene $\mathbb{R}^2$
Hier werden die drei Typen *point*, *segment* und *line* genutzt. Ein Segment ist die Strecke zwischen zwei Punkten und *line* die Gerade durch zwei Punkte.

Beispiel 6.1
point $p(17.5, 1.3)$
point $q(x, y)$
$p \neq q$

segment $s(p, q)$



line $l(p, q)$



6.1 Operationen

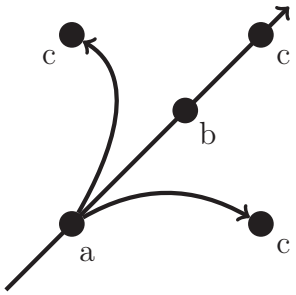
xy-Ordnung: $q \leq p$

$$p.comp_xy(q) = \begin{cases} -1, & p \leq_{xy} q \\ 0, & p = q \\ +1, & p \geq_{xy} q \end{cases}$$

Zuerst werden die x-Koordinaten der beiden Punkte verglichen. Sind diese gleich, werden die y-Koordinaten verglichen.

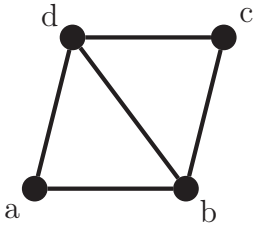
6.2 Orientierung

3 Punkte a, b, c



$$\text{Operation}(a,b,c) = \begin{cases} -1, & a,b,c \text{ Rechtskurve} \\ 0, & a,b,c \text{ co-linear} \\ +1, & a,b,c \text{ Linkskurve} \end{cases}$$

6.2.1 Lineare Algebra



Fläche des Parallelogramms hat Vorzeichen

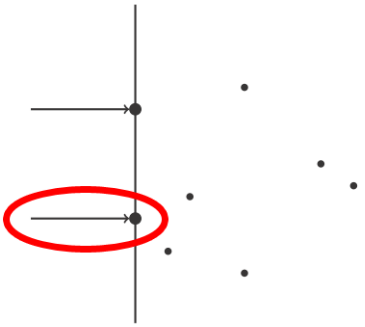
$$\begin{vmatrix} a_x & a_y & 1 \\ b_x & b_y & 1 \\ c_x & c_y & 1 \end{vmatrix} \quad \begin{array}{ll} > 0, & a,b,c \text{ linksorientiert} \\ < 0, & a,b,c \text{ rechtsorientiert} \\ = 0, & a,b,c \text{ co-linaer} \end{array}$$

Fläche $\Delta = |\cdot|$

$$\text{Operation}(a,b,c) = sign \begin{vmatrix} a_x & a_y & 1 \\ b_x & b_y & 1 \\ c_x & c_y & 1 \end{vmatrix} \rightarrow \text{einfach arithmetische Ausdruck}$$

6.2.2 Minimum / Maximum in xy-Ordnung

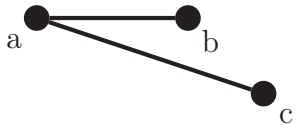
Min_{xy} ist der am meisten links liegende Punkt. Existieren mehrere, dann ist es der



$\rightarrow$ Extrempunkt

Analog: Max_{xy}, cmp_{xy}, ...

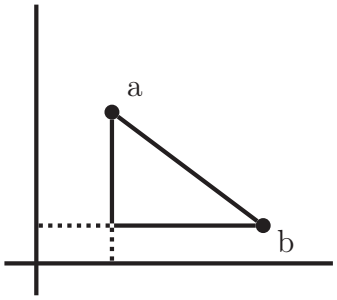
6.2.3 Vergleich von Entfernungen



dist(a.b) ? dist(a,c)
Ist a weiter von b oder von c entfernt?

dist Euklidischer Abstand
→ L_2-Norm

$dist(a, b) =$
$d_x \leftarrow a_x - b_x$
$d_y \leftarrow a_y - b_y$
$\sqrt{d_x^2 + d_y^2}$ (Hypothenuse)



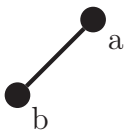
cmp_dist$(a, , b, c)$:= sign(dist(a,c) - dist(a,b))
$\sqrt{d_x^2 + d_y^2}$ - $\sqrt{d_x'^2 + d_y'^2}$
→ Vergleiche $(dist(a, c))^2$ mit $(dist(a, b))^2$
$= sign((c_x - a_x)^2 + (c_y - a_y)^2 - ((b_x - a_x) + (b_y - a_y)^2))$
→ Eliminierung der Quadratwurzel

Nur $+, -, *$
Vorteil:

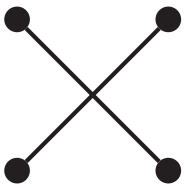
- schneller
- exakt (z.B. für int/long)

6.2.4 Anwendung von orientation (→ Schnitt-Test)

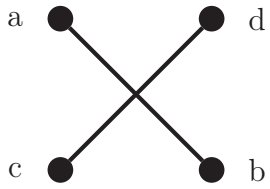
segment $s(a, b)$




```
point s.start();
point s.end();
```
Schnitt $s_1 \cap s_2$



Test mit orientation



Schnitt $\Leftrightarrow$
orientation(a,b,c) $\neq$ orientation(a,b,d)
$\wedge$ orientation(c,d,a) $\neq$ orientation(c,d,b)

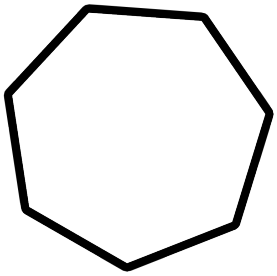
6.2.5 Problem: Konvexe Hülle

Eingabe: Liste von Punkten `list<point> L;`

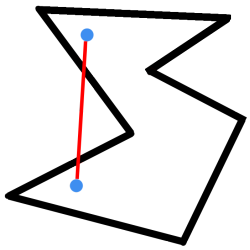


Ausgabe: kleinstes Polynom P, das alle Punkte aus L enthält.

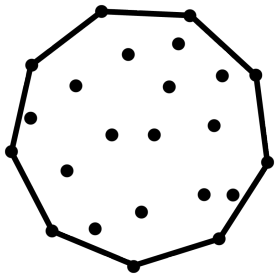
Konvex



Nicht konvex



Menge S kovex $\Leftrightarrow \forall\ a, b \in S : \overline{ab} \subseteq S$



Man kann es sich wie ein Gummiband vorstellen, das außen um eine Menge an Nägeln gelegt wird.

Menge S kovex $\Leftrightarrow (a, b \in S : \overline{ab} \subseteq S)$
$\rightarrow$ Polygon $p = CH(L)$

```
list<point> ConvexHull(list<point>& L){
   list<point> CH; // Resultat
   // Startpunkt (Ecke) -> lineare Suche
    point q0 = L.first();
    point p;
    forall(p,L){
      if(p.cmp_xy(q0) == -1){
         q0 = p;
      }
   }
   CH.append(q0);
   L.del(q0);
   while(true){
      point q = L.first()
      forall(p,L){ // lineare Suche nach max. Winkel
         if(CH.last().orientation(q,p) == -1
            || ((CH.last().orientation(q,p) == 0)
            && CH.last().cmp_dist(q,p) == -1)){
            q = p;
         }
      }
      L.del(q);
      if(q == q0) break;
      CH.append(q);
   }
   return CH;
}
```

orientation(p,q,r) und

$$cmp_xy(a,b) = \begin{cases} +1 & a >_{xy} b \\ 0 & a =_{xy} b \\ -1 & a <_{xy} b \end{cases} \text{ und}$$

$$cmp_dist(a,b,c) = \begin{cases} +1 & dist(a,c) > dist(a,b) \\ 0 & dist(a,c) = dist(a,b) \\ -1 & dist(a,c) < dist(a,b) \end{cases} \text{ stehen zur Verfügung.}$$

Laufzeit (Output sensitiv)
Schritt 2 kostet $\mathcal{O}(n)$.
$\mathcal{O}(h * n)$ wobei $h = \#Ecken$ von $CH(S)$
Für jede Ecke 1x lineare Suche

Worst-case: h=n (oder Bruchteil von n)
$\rightarrow \mathcal{O}(n^2)$ z.B Kreis oder konvexe Kurve (z.B. Parabel)

Best-case: h = konstant z.B. h=3

Bessere Laufzeit
$\mathcal{O}(n * log_2(n))$ ($\rightarrow$ Sortieren)

Diese Laufzeit ist optimal, da das Finden einer konvexen Hülle (CH) mindestens so schwer ist, wie das Sortieren von n verschiedenen Zahlen.

Beweis Reduktion von Sortieren auf CH
Verschiedene Algorithmen mit Laufzeit $\mathcal{O}(n * log_2(n))$

Strategie:

i) Inkrementell

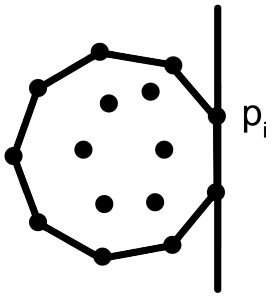
ii) Divide & Conquer

iii) Scan / sweap

□

Inkrementeller Algorithmus

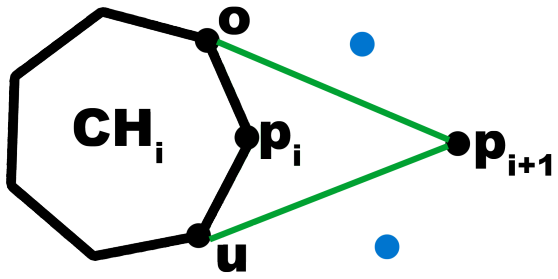
1.) Sortiere die Punkteliste nach xy-Ordnung $\rightarrow$ Folge von Punkten ($\mathcal{O}(n * log_2(n))$)
Idee: Durchlaufe die Punkte $\rightarrow$ aktueller Punkt p_i $(i = 1, \ldots, n)$



$CH_i = CH(p_1, \ldots, p_i)$ ist berechnet.

Schritt $i \to i+1$

Da p_i extrem ist in $\{p_1, \ldots, p_i\}$ ist p_i (rechteste) Ecke von CH_i.



Die blauen Punkte sind mögliche Stellen, an denen p_{i+1} auch liegen kann.

intuitiv:

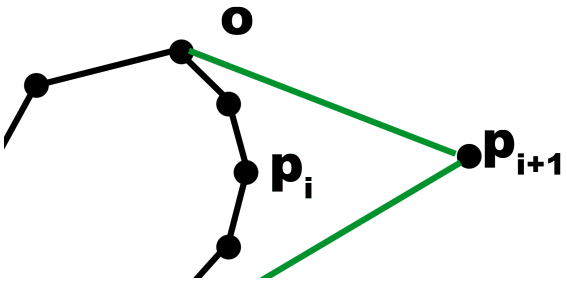
i) Entferne alles (Ecken), das von p_{i+1} sichtbar (beleuchtet) ist, aus CH_i.

ii) Füge p_{i+1} stattdessen ein

formaler:

i) Finde die Tangentenberührpunkte o (oberer Berührpunkt) und u (unterer Berührpunkt) und entferne alle Ecken zwischen o und u.

ii) Füge p_{i+1} unmittelbar nach u ein (vor o)



Die Ecke, an der es das erste Mal einen Links-Knick von p_{i+1} aus, wird als oberer Berührpunkt genommen.

Implementierung

i) Datenstruktur für CH: zyklische Liste L von Punkten.
list<point> L, CH;
L.cyclic_succ(p) zyklischer Nachfolger
L.cyclic_pred(p) zyklischer Vorgänger
Punkte in L sind gegen den Uhrzeigersinn sortiert.

ii) Initialisierung: Dreieck aus p_1, p_2, p_3
Annahme: Nicht colinear (orientation$(p_1, p_2, p_3) \neq 0$
korrekt orientiert: oruientation$(p_1, p_2, p_3) > 0$

iii) Schleife für $p_1, \ldots, p_n$

```
for(i=4; i≤ n; i++){
   // oberer Berührpunkt
    p ← p_{i-1};
   while(orientation(p_i,p,CH.cyclic_succ(p)) ≤ 0){
      // Für Triangulierung
      "Gibt Δ(p_i,p,CH.cyclic_succ(p)) aus";
      p ← CH.cyclic_succ(p);
   }
   o ← p;
   // unterer Berührpunkt
   p ← p_{i-1};
   while(orientation(p_i,p,CH.cyclic_pred(p)) ≥ 0){
      // Für Triangulierung
      "Gibt Δ(p_i,p,CH.cyclic_pred(p)) aus";
      p ← CH.cyclic_pred(p);
   }
   u ← p;
   // Entferne alle Punkte dazwischen
   while(CH.cyclic_succ(u) ≠ o){
      CH.delete(CH.cyclic_succ(u));
   }
   // Füge p_i nach u ein
   CH.insert_after(p_i,u);
}
```

Analyse

1) Sortieren $\mathcal{O}(n * log(n))$
2) for-Schleife
   Beobachtung: Jeder Punkt wird genau einmal in die Liste CH eingefügt und höchstens einmal entfernt
   $\Rightarrow$ for-Schleife hat Laufzeit $\mathcal{O}(n)$

Laufzeit $\mathcal{O}(n * log(n))$
$\mathcal{O}(n)$, falls die Punkte sortiert sind

Divide & Conquer

$p_1, \ldots, p_n$ sortiert.

```
CH(p_1, ..., p_n){
    if(m≤ 3){
        Konstruiere Dreieck
    }
    else{
        m ← ⌊ n/2 ⌋;
        A ← CH(p_1, ..., p_m);
        B ← CH(p_{m+1}, ..., p_n)
    }
}
```

```
    bool empty(); // liefert { true   falls Stack leer
                             { false  sonst
    /* Alle Operationen haben Laufzeit O(1). Der Platzbedarf ist
    proportional zur Anzahl der Elemente in Stack.*/
}
push: int_stack ×int→int_stack
pop: int_stack×int_stack⇝int_stack×int
top: int_stack⇝int
```

4.05.09

1.1.2 Bemerkungen zur Klassendeklaration in C++ (*int_stack*)

Konstruktor *int_stack()*, realisiert *create* →*int_stack*

- hat gleichen Namen wie die Klasse
  - Kann auch Argumente haben, z.B. *int_stack(int sz)* ←maximale Größe *(create:int→int_stack)*
  - kein Resultat
  - wird automatisch aufgerufen bei der Deklaration einer Variable des Datentyps *int_stack S;* mit Argumenten *int_stack*

  Parameterübergabe an Konstruktor auch mit =

Beispiel

```
string x("hallo");
string x = "hallo";
```

Destruktor ~*int_stack();*

- ~ + Klassenname, keine Argumente
  - automatisch aufgerufen, wenn Gültigkeitsbereich (Lebensdauer, Scope) einer Varibale des Datentyps endet (am Ende des Blocks und der Variable deklariert wurde).

```
{ int_stack S,
    ...
    s.push(17);
    ...
} //←Aufruf von ~ int_stack() für Variable S
```

  - wird hauptsächlich verwendet, um belegten Speicherplatz freizugeben.

1. Bei den "normalen" Methoden (*push, pop*, ...) erscheint das Objekt (s), auf dem die Operation ausgeführt wird, nicht in der Parameterliste. Es wird implizit übergeben (ist aber bei der Implementierung über den sog. "*this*"-Pointer verfügbar).
   Zum Aufruf der Operation verwendet man die Memberschreibweise

   ```
   S.push(17) // (statt push(S,17) in sprache "C")
   ```

   Operationen heißen auch Memberfunktionen oder Methoden.

2. Die Operationen "*top*" und "*empty*" verändern ihr Objekt nicht.
   Dies kann man mit dem Stichwort *const* hinter der Parameterliste anzeigen.

   ```
   int top() const;
   bool empty() const;
   ```

   Der Compiler nutzt diese Hinweise, um *const*-Regeln zu überprüfen. Insbesondere dürfen auf Konstanten des Datentyps nur *const*-Operationen aufgerufen werden (später mehr hierzu).

3. Parameter
   Normale Parameterübergabe by value (nicht Referenz →JAVA)

   ```
   S.push(x) // Kopie von x wird auf stack S gepusht.
   ```

   (→Kontrolle der Art der Parameterübergabe, später)

1.2 Implementierung von Datentypen

1.2.1 Implementierung von Stacks

(im Gegensatz zur Spezifikation)

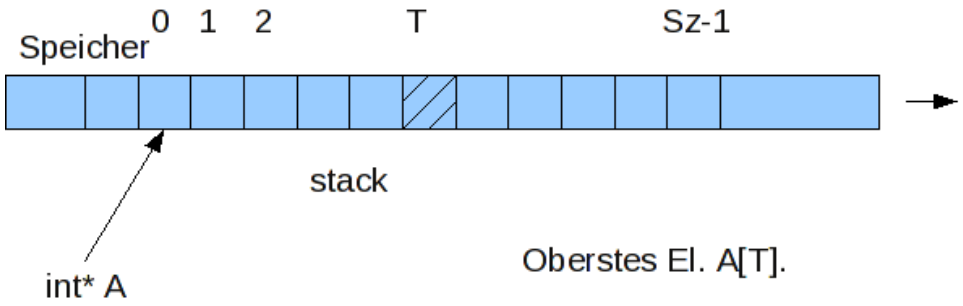
Array-Implementierung. Datenstruktur besteht aus

1. Feld ganzer Zahlen
   a) Verweis auf oberstes Element
   b) Größe des Feldes

   ```
   class int_stack {
       ...
   private: int* A; // Feld (Pointer auf 1. El)
            int T;  // Top-Index
            int sz; //Größe des Feldes
   ```

   Pointer, Felder, Referenzen in C/C++ bitte anschauen.

Abbildung 2: Array-Implementierung



INVARIANTE der Datenstruktur

1. A ist Feld von sz Elementen.
2. $-1 \leq T \leq \text{sz} - 1$
3. Inhalt des Stacks ist dann $A[0], A[1], ..., A[T]$, wobei $A[T]$ das oberste Element (top).
4. Der Stack ist leer genau dann wenn $T = -1$

Anlegen eines Feldes

```
int* A = new int[100];
A[7]⟺*(A+7)←Zugriff über Pointer
```

```
Freigabe: delete[] A;
```

1.2.2 Implementierung der Operationen (Methoden)

Die in der Klasse deklarierten Methoden werden nun definiert.

Syntax

```
Resultatstyp, Klassenname :: Operation(Parameterliste) const
{
    Rumpf
}
```

Konstruktor

```
int_stack::int_stack()
{ //initialisiere Datenstruktur
```

```
        sz = 100 //am Anfang Feld der Größe 100 (INVARIANTE)
        A= new int[sz];
        T = -1; //leerer Stack

    }
```

Destruktor

```
    int_stack ::~int_stack() // pop-Operation
    { delete[]A; }
```

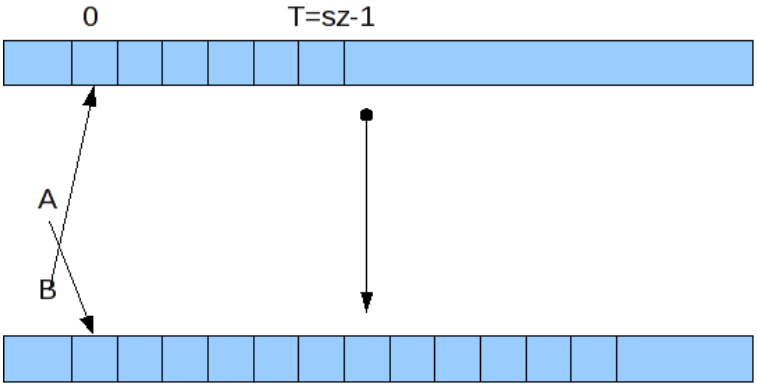
push-Operation: verdoppelt Feld, falls es zu klein ist

***void**: int_ stack::push(int x)*

```
    void: int_stack::push(int x)
    {_
        if (T+1 == sz)
        { // verdopple Feld
            int* B = A;
            sz *= 2;
            A = new int [sz];
            for (int i = 0; i <= T; i++)
                A[j] = B[j];
            delete[] B;
        }
        A[++T]=x;

    }
```

Abbildung 3: Verdopple Feld



***pop*-Operation:** nur auf nicht-leeren Stack (Precondition)

```
int int_stack::pop()
{
    if (T == -1)
    {
        EXCEPTION (‘‘pop auf leerem Stack’’);
    }
    return A[T--];
}

bool int_stack::empty() const

bool int_stack::empty() const
{
    return T == -1;
}
```

11.05.09

Dateien *int_stack.h* →Deklaration der Klasse (Spezifikation)
int_stack.cpp →Implementierung der Operatoren (Methoden)

Anwendungsprogramm

```
#include <int_stack.h>
...
int_stack S;
S.push(17);
...
```

C++ inline Funktionen Bei kleinen Funktionen z.B. *s.empty()* ist Aufwand für Aufruf und Rückkehr oft viel größer als die tatsächlichen Kosten der Funktion.

Dann ist es viel effizienter den Rumpf an der Aufrufstelle (als Programmtext) einzukopieren, statt einen wirklichen Funktionsaufruf auszuführen. (vgl. mit Makro-Konzept in C).

⇒Die Definition der Inline-Funktion muss in *int_stack.h* (HeaderDatei) stehen!

2 Möglichkeiten (in h-Datei)

Nach Klasse

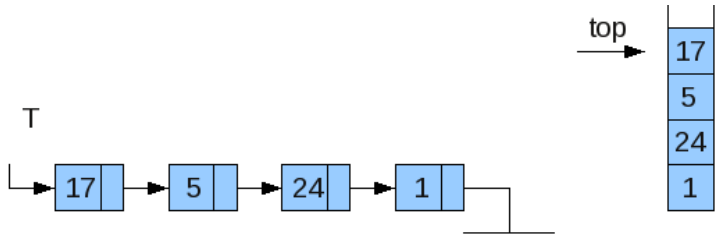
```
inline bool int_stack::empty() const
    { return T == -1; }
```

Schreibe den Rumpf direkt hinter die Deklaration der Methode in der Klasse

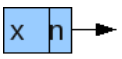
```
class int_stack {
    ...
        public
        (inline) bool empty() const {return T == -1; };
```

1.2.3 Implementierung 2: Listen

Abbildung 4: Liste




```
class int_stack { //←int_stack.h
    class stack_elem;
    {
        int vol;
        stack_elem* netx;
        public
        stack_elem(int x, stack_elem* n)
        { val = x; next = n; } //Konstanter
        stack_elem(int x, stack_elem* n): val(x),next(n){} //alternativ
        //besser
    };
```




```
    int_stack.cpp
    //Konstruktor
```

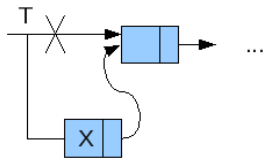



```
        int_stack::int_stack ()
        {T = NULL;}
        bool int_stack::empty() const
        { return T ==s NULL; }
        void int_stack:: push (int x)
        { //vorne einfüge
            T= new stack_elem (x,T);}
```

```
//Destruktor: muss alle Listenelemente mit delete freigeben
```

→Vollständige Implementierung: Übung.

Abbildung 5: das Einfügen



Laufzeit

Listenimplementierung: O(1) pro Operation worst-case!

Array: O(1) amortisiert (O(n) worst-case!)

In der Praxis ist die Array-Implementierung meistens schneller, weil sie die Daten kompakt (lokal) im Speicher ablegt →sehr gutes Cache-Verhalten! (später mehr dazu)

Korrektheit unserer Implementierung Zunächst müssen wir definieren, was Korrektheit bedeutet.

Eigentlich haben wir jeweils 2 Datentypen.

1. den abstrakten Datentyp *stack* (durch die Deklaration der Klasse in *stack.h*)
2. den konkreten Datentyp durch die Definition der privaten Daten und der Methoden (*stack.cpp*).

Wir müssen zeigen, dass diese beiden Typen äquivalent sind.

1.2.4 Array-Implementierung

abstrakter Zustand Folge von *int*'s.

konkreter Zustand Inhalt der Variablen A, sz, T

In der Implementierung garantieren wir, dass nicht alle Kombinationen von T, sz, A möglich sind, sondern nur die mit

1. A ist Feld der Länge sz (siehe INVARIANTE)
2. $-1 \leq T < sz$

→gültige korrekte Zustände Z

Um die Korrektheit zu zeigen

1. definiere eine Abbildung F von den korrekten Zuständen Z in die Folge der abstrakten Zustände S mit $\overbrace{A, sz, T}^{Z} \xrightarrow{F} \begin{cases} \text{Folge A[0], ... , A[T]} & \text{falls} T \geq 0 \\ \text{leere Folge} & \text{sonst} \end{cases}$
2. Jede Operation *op* des abstrakten Datentyps wird realisiert durch Methode f_{op} des konkreten Typs

   s ist abstsrakter Zustand, d.h. $s \in S$.

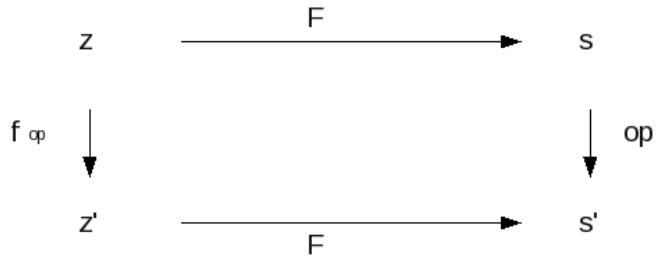
   Beispiel $\mathsf{push}(s, x) \rightarrow s'$

   $\mathrm{f}_{\mathrm{push}} \overbrace{A, sz, T}^{Z}, x \rightarrow A', sz', T'$

   (z ist konkreter gültiger Zustand $z = A, sz, T$)
3. Zeige, dass für jede Operation *op* und gültigen konkreten Zustand z gilt.

   $F\left(f_{op}(s)\right) = op\left(F(z)\right)$

Abbildung 6: kommutatives Diagramm



4. Zeige, dass die Konstruktoren

   a) gültige konkrete Zustände erzeugen

   b) die Create-Operation realisieren

beim Stack F (A,sz,T) = A[0], ...,A[T]

18.05.09

Korrektheit der int_stack Implementierung →Übung

dazu:

1. zeige F(für alle Operationen
2. zeige, dass Konstruktoren gültige konkrete zustände erzeugen

Zur Vervollständigung der Datentypenparameter (Resultatsrückgabe) und Wertzuweisung in C++ grundsätzlich 2 Arten:

Übergabe "by value". Es wird eine Kopie des Objektes an die Funktion übergeben.

Syntax: F(T x) z.B. F (int i)

```
void int_stack::push (int x)
```

Aufruf F(y), dann wird im Rumpf von F der formale Parameter x mit einer Kopie des aktuellen Parameters initialisiert. Änderungen des Wertes vom formalen Parameter x haben einen Einfluss auf den Wert von y.

Übergabe "by reference" Syntax: F(T & X) z.B. F(int & i)

Es wird das Objekt selbst übergeben (Referenz).

Aufruf F(y).

Im Rumpf bezeichnet der formale Parameter y dasselbe Objekt im aktuellem Parameter x (keinefalls eine Kopie) d.h. Änderung vom y ändert auch x.

Beispiel

```
void cmp_and_swap(int & a, int & b)
{
    if(a>b)
    {
        int tmp = a;
        a = b;
        b = tmp;
    }
}
```

Aufruf

```
int x = 17; int y = 7;
cmp_and_swap(x,y)
```

Variante "const reference" Es wird das Objekt selbst übergeben, aber im Rumpf der Funktion wie eine Konstante behandelt (Wert darf nicht verändert werden)

1. nur const-Methoden
2. keine Wertzuweisung
3. mit Übergabe by Value oder const

Syntax: F (const T & x)

Anwendung Rumpf ändert den Wert von x nicht

- by value Übergabe zu teuer (Kopie)

Rückgabe von Resultaten

1. by value

```
T F( ... )
{
    T x;
    ...
    return x;
}
```

Aufruf `y = F(...) Kopie der lokalen Variable x im Rumpf`

2. by reference

```
T & F (...)
```

Beispiel

```
int max (int & a, int & b)
{ return (a > b) ? a:b; }
x = 17, y = 7;
max (x, y) = 13;
//variable (L-value)
```

Array-Klasse

```
class int_array
{
    int & elm (int i ) { ... }
    A.elem(i) = 17;
```

```
}
```

später: A [i] →Überladen von Operatoren.

3. const int & max (const int & a, const int & b)

Definition der “by value” Übergabe für Klassen-Form

→copy Konstruktor

Wenn nichts anders gesagt wird, dann werden Klassenobjekte bitweise kopiert (jedes Datenfeld).

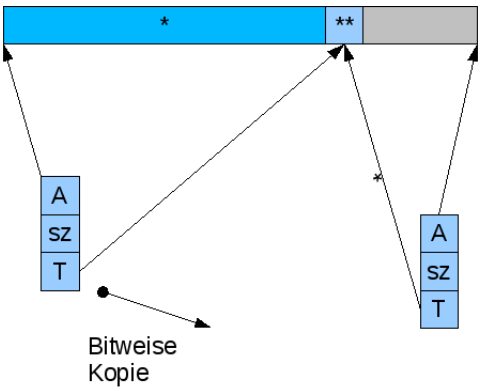
Beispiel: int_stack

```
void F (stack s)
{
    s.pop(); *
    s.push (17); **
}
```

Aufruf stack S;

F(S”)

Abbildung 7: Aufruf von stack S



⇒Aufruf: F(s’) verändert s’

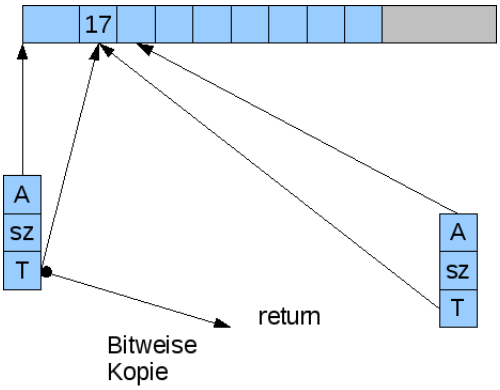
Resultatrückgabe

Beispiel

```
int_stack F (int x)
{
```

```
    int_stack S;
    S.push (x)
    return S;
}
int y = F(17).top();
```

Abbildung 8: push-Operation



Konstruktor von int_stack gibt Feld am Ende von F für d.h. Feld existiert nicht außerhalb von F - Widerspruch!

⇒Bitweise Kopie reicht für Komplexe Klassen nicht aus

Lösung in C++ Definition eines copy-Konstruktors der immer dann verwendet wird, wenn eine Kopie gemacht werden muss (bei Übergabe)

Syntax für Klasse T

```
class T
{
    T (const T & x);
    /*erzeugt eine Kopie von x
    durch entsprechende Initialisierung
    der Datenfelder von T.
```

für int_stack

```
class int_stack { const int_stack & s )
{
    sz = S.sz;
    T = S.T;
```

```
        A = new int [sz];
        for (int i = 0; i < sz; i++)
        {
            A[i] 0 S.A[i];
        }
    }
```

Wertzuweisung x = y;

Default: x wird bitweise durch y überschreiben

Probleme

1. wie beim copy-Konstruktor
2. altes Objekt wird nicht zerstört

Destruktor

Lösung in C++ Definiere einzelne Wertzuweisungs-Operatoren →allgemein überladen von Operatoren (nachlesen!)

25.05.09

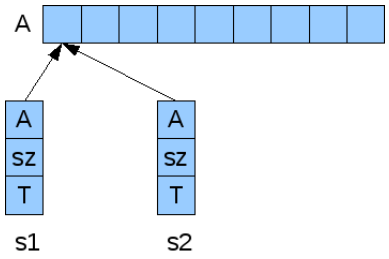
Wertzuweisung: x = y;

Default: Datenfelder von x werden bitweise durch die Datenfelder von y überschrieben.

Gleiches Problem wie beim copy constructor in_stack s1, int_stack s2

s1 = s2; ⇒

Abbildung 9: s1=s2



zeigen auf dasselbe Feld.

Lösung: Überladen des "=" Operators, ...

(→Übung Nachher wie Operatoren in c++ überladen werden)

Syntax: Memberfunktion für eine Klasse T

```
class T
{
    public
    T & operator = (const T &);
```

Aufruf: T x,y; x = y; wird vom Compiler durch x.operator=(y); ersetzt.

$a = b = c; \Rightarrow a = (b = c); \Rightarrow$a.operator=(b.operator=(c));

Implementierung von x.operator = (y) muss

1. x zerstören (wie im Destruktor)
2. Kopie von y erzeugen (wie Copy-Konstruktor)

im Beispiel:

```
class int_stack
{
    puclic: int_stack & operator =(const int_stack);
    int_stack & int_stack :: operator = (const int_stack S)
    {
        delete[] A; //Zerstörung
        sz = S.sz;
        T = S.T;
        A = new int [sz]
        for ( int i = 0; i < sz; i++) A[i] = S.A[i];
        return *this;
    }
```

Problem: S=S; delete [] A serstört S.

Einfache Lösung: teste, ob this == & S;

⇒1. Zeile des Operators:

```
if (this == & S) return * this;
```

andere wichtige Operatoren (je nach Typ):

- Test auf Gleichheit

```
class T
{
    bool operator == ( const T & x);
    bool operator < ( const T & x);
    bool operator > ( const T & x);
    bool operator != ( const T & x);
    ...
```

```
}
```

- arithmetische Operatoren: +, -, *, / ...
- spezielle Operatoren:
  - Feldzugriff: operarot []
  - Funktionsaufruf operator ()

  Übung: Ein/Ausgabe in C++ →streams nachlesen!
- I/O-Operatoren << (Ausgabe), >> (Eingabe)

Beispiel: Konsolen I/O

```
cin >> x; // Eingabe
cout << x << endl; // Ausgabe
```

2 Parametrisierte Datentypen oder generische Datentypen

d.h. verwendbar für beliebige Elementtypen

Beispiel: stack von int, double, string, ...

2 Strategien:

1. Ableitung (Vererbung)
2. Templates

Ziel: Wiederverwendung von Programm-Code.

2.1 Vererbung und dynamisches Binden

Situation Man braucht einen Datentyp A der ähnlich zu einem bereits definierten Typ B (→Klasse) ist. A soll

- einen Teil der Daten/Operationen von B wiederverwenden
- andere hinzufügen
- einige verändern (auf andere Weise implementieren)

Beispiel: →Spezialisierung Datentyp Polygon existiert, wir wollen Datentyp Rechteck definieren.

Da jedes Rechteck ein Polygon ist ("ist ein" Relation), können alle Polygon-Operationen übernommen werden. Einige Operationen können effizienter implementiert werden. (z.B. Flächeninhalt), andere sind nur für Rechtecke definiert (z.B. Seitenverhältnis).

Zusammenfassung von Algorithmische Geometrie.

Kapitel 0: Vorbemerkungen:

Inhalt der Vorlesung: Algorithmen und Datenstrukturen zur Lösung geometrischer Probleme.
Beispiele: Robotik / Bewegungsplanung, Computergrafik, CAD, VLSI
Typische Probleme:
- Konvexe Hülle
- Zerlegung in einfache konvexe Teile (zB. Triangulierung).
- Schnittprobleme (zB. Segmente, Polygone)
- Suchprobleme (Memberanfrage in Pktmenge, Nearest-Neighbor, Range-Abfragen).
- Bewegungsplanung (Roboter)
- Hidden line / Surface (Elimination).

Grundlegende Ansätze bzw. Vorgehensweisen:
- Devide & Conquer
- Plane Sweep.
- Hierarchische Darstellung
- Dualität
- Randomisierte Algorithmen.
- Rundungsfehler u. degenerierte Eingaben.

Kapitel I: Konvexe Hüllen.

1.1. Konstruktion der konvexen Hülle einer Pktmenge im $\mathbb{R}^2$ – Grundlagen.

1.1.1. Def: Sei $S \subset \mathbb{R}^2$ Pktmenge.
S heißt konvex: $\Leftrightarrow \forall p,q \in S$ gilt: $\overline{pq} \subset S$.
konvexe Hülle einer Menge $S \subset \mathbb{R}^2$ ($CH(S)$) ist die kleinste konvexe Menge, die S enthält.

Wir betrachten nun endliche Pktmengen, dh. $|S| = n \in \mathbb{N}$.

Anm: Der Rand von $CH(S)$ ist ein konvexes Polygon mit Ecken aus S.
Bew. Übung.

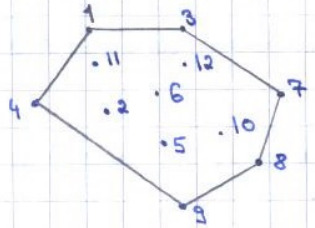
Konvexe Hülle-Problem für endl. Menge $S \subset \mathbb{R}^2$:
Berechne die Folge der Ecken von $CH(S)$ gg. den Uhrzeigersinn (pos. orientiert).
$\rightarrow q_1, \dots q_\ell \in S$. (Anordnung definiert Kanten der Fläche).
Degenerierte Fälle: $\ell = 2$ (alle Pkte sind colinear)
$\ell = 1$ (alle Pkte in S sind gleich).

Problem im $\mathbb{R}^3$ analog, $CH(S)$ ist konvexes Polyeder mit Ecken aus S.

1.1.2. Satz: Berechnung der konvexen Hülle von n Pktn. im $\mathbb{R}^2$ ist mind. so schwer wie das Sortieren von n Zahlen.
Bew. siehe Üb. (Idee: Führe Sortieren von $x_1 .. x_n$ zurück auf $CH(\{p_1 .. p_n\})$.).

1.1.3. Korollar: Im Allgemeinen braucht die Berechnung von $CH(S)$ Zeit $\Omega(n \log n)$.

1.1.4. Bsp:



$S = \{1..12\}$, $CH(S) = 8,7,3,1,4,9$
(Jedes zyklische Shift)

1.1.5. Vereb: Lexikografische Ordnung (siehe Strings) für Pkte im $\mathbb{R}^2$:
Lexikogr. Sortierung nach kartesischen (x,y)-Koordinaten, dh. für zwei Pkte in der Ebene, also $p,q \in \mathbb{R}^2$ gilt: $p < q :\Leftrightarrow p_x < q_x \vee (p_x = q_x \wedge p_y < q_y)$
Dh. Sortierung von links nach rechts bzw. von unten nach oben (bei gleicher x-Koordinate).

1.2 Algorithmus I: Gift Wrapping.

Intuitiv: Punktmenge wird mit Geschenkpapier eingewickelt.

Zunächst: $S \subset \mathbb{R}^2$. Voraussetzung: $|S| < \infty$

Geo: $CH(S) = p_1 .. p_n$.

1.2.1. Idee:

1) Startpkt p_1 : Pkt mit der kleinsten y-Koordinate.
(Falls mehrere wähle den linkesten. DH. Minimum in p_x - lexikogr. Ordnung.
(Übung : p_1 ist Ecke der Konvexen Hülle).

2) Wie findet man p_2? (Nachfolger von p_1 auf Konv. Hülle).
Betrachte horizontalen Strahl ℓ durch p_1 (nach rechts). Drehen den Strahl gg. den Uhrzeigersinn bis wir einen weiteren Pkt von S treffen. (Ann. $|S| > 1$).
So erhalten wir p_2.
Genauer: $\forall q \in S \setminus \{p_1\}$ sei α_q der Winkel zwischen ℓ und $\overline{pq}$.
Wähle p_2 so, dass α_{p_2} minimal. Falls mehrere Pkte mit minimalen Winkel, wähle den Pkt mit maximaler Entfernung zu p_1.
$\rightsquigarrow$ lineare Suche in S nach Minimum bzgl. oben definierter Ordnung.

3) Setze Algor. bei p_2 fort, d.h. ℓ wird nun der Strahl, der
- in p_2 beginnt.
- in Richtung p_1p_2 zeigt.

$\rightsquigarrow$ Suche Minimum in $S \setminus \{p_1, p_2\}$.

4) Wiederhole bis p_1 wieder erreicht wird.

1.2.2. Korrektheit : Übung.

1.2.3. Laufzeit:

$CH(S) = p_1 .. p_R$, $|S| = n$.

p_1: lineare Suche in S kostet $O(n)$ (lin. Suche nach Min. bzgl. p_y)

$p_2 .. p_R$: lin. Suche in S kostet $O(n)$ (lin. Suche nach Winkelordnung).

$\Rightarrow$ Gesamtlaufzeit: $O(R \cdot n)$.

1.2.4. Satz:

Sei S eine Menge von n Pkten im $\mathbb{R}^2$ und R die Zahl der Ecken der Konvexen Hülle $CH(S)$. Dann kann $CH(S)$ in Zeit $O(R \cdot n)$ berechnet werden. Bew. siehe oben.

1.2.5. Bem:

$R \in \{1..n\}$

Worst Case: $R = n$ (zB. alle Pkte aus S liegen auf einem gemeinsamen Kreis, das Innere ist leer). $\Rightarrow$ Laufzeit: $O(n^2)$.

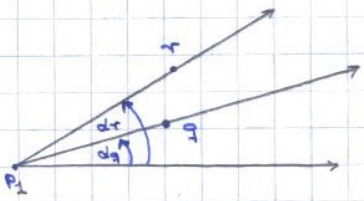
Bester Fall: $R = \text{const.} \Rightarrow$ Laufzeit: $O(R \cdot n) = O(n)$.

1.2.6. Details der Implementierung:

1) Wie findet man Pkt q so, dass α_q minimal ist?

$\rightsquigarrow$ Wir müssen Winkel vergleichen.

```
q ← undefiniert
α_q ← ∞
forall p ∈ S \ {p_1} do
    if α_p < α_q then
        q ← p
        α_q ← α_p
    fi
od
```



Im Allg. 3 Pkte p, q, r $\rightarrow$ vergleiche Winkel

Naiv: Berechne Winkel aus Koordinaten der Pkte mit trigonometrischen Fktnen.

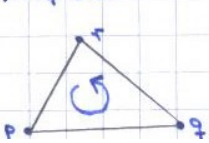
$\Rightarrow$ Nachteile:
- Langsam
- Präzisionsprobleme
- Robustheitsproblem (Definitionsbereich der trigonometrischen Fktnen ist eingeschränkt).

$\Rightarrow$ Besser: Andere Betrachtungsweise:

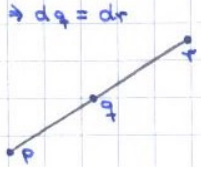
Dazu betrachte Dreieck p, q, r

Drei mögliche Orientierungen:

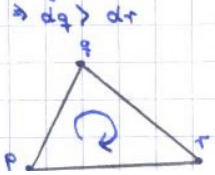
a) positiv orientiert (left-turn) $\Rightarrow \alpha_q < \alpha_r$



b) Orientierung 0 (collinear) $\Rightarrow \alpha_q = \alpha_r$



c) negativ orientiert. (right-turn) $\Rightarrow \alpha_q > \alpha_r$



Allgemein: Seien $p=(p_x,p_y)$, $q=(q_x,q_y)$ und $r=(r_x,r_y)$ drei Pkte im $\mathbb{R}^2$.
Sei $\vec{p}$ der Vektor, der vom Nullpkt ausgeht und in Richtung p zeigt. Und seien $\vec{q}$ und $\vec{r}$ Vektoren, die von p ausgehen und in Richtung q bzw. r zeigen.
Betrachte nun das Parallelogramm, welches durch die Vektoren $\vec{q}$ und $\vec{r}$ aufgespannt wird. Sei $V(P)$ = die Fläche des Parallelogramms.
Beh.: $V(P) = |\det A|$, wobei $A = \begin{pmatrix} q_x-p_x & q_y-p_y \\ r_x-p_x & r_y-p_y \end{pmatrix}$

Bem.: diese Beh. überträgt sich auch auf $\mathbb{R}^n$.
Man betrachte hierbei $u_1..u_n \in \mathbb{R}^n$ und $P = \{u_1a_1+..+u_na_n : 0\le a_i\le 1, i\in\{1..n\}\}$.
$\Rightarrow V(P) = |\det A|$, A entsprechend.

Wir betrachten Δpqr:

1.2.6.1 Satz: $p,q,r\in\mathbb{R}^2$, $A = \begin{pmatrix} q_x-p_x & q_y-p_y \\ r_x-p_x & r_y-p_y \end{pmatrix}$.

$\Rightarrow$ (i) $|\det A|$ = 2. Fläche von Δpqr.
(ii) $\operatorname{sign}(\det A)$ gibt die Orientierung an:
-1: neg. orientiert (right turn)
0: collinear.
$+1$: pos. orientiert (left turn).

Beweis siehe Vorordner.

1.2.6.2 Def: Wir definieren das geometrische Prädikat:

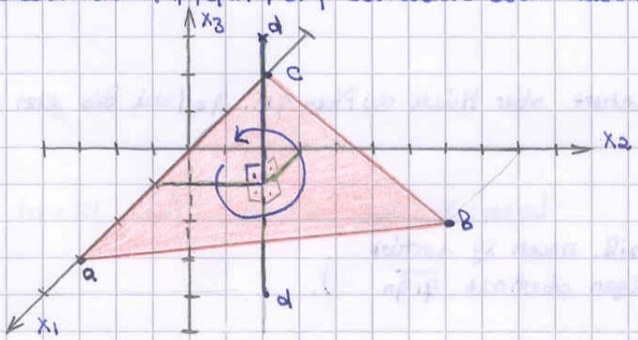
$$\text{orientation}(p,q,r) = \operatorname{sign}\begin{vmatrix} p_x & p_y & 1 \\ q_x & q_y & 1 \\ r_x & r_y & 1 \end{vmatrix} \left(= \operatorname{sign}\big((q_x-p_x)(r_y-p_y)-(r_x-p_x)(q_y-p_y)\big)\right.$$

Dh. orientation(p,q,r) und damit der Test $d_r < d_q$ kann mit zwei Multiplikationen und fünf Subtraktionen berechnet werden.

2) Falls $d_r = d_q$ (dh. orientation$(p,q,r)=0$), müssen wir herausfinden, welcher der Pkte q oder r weiter von p entfernt liegt.
Naiv: Berechne Distanzen (mit eukl. Metrik):
$\text{dist}(p,q) = \sqrt{(p_x-q_x)^2+(p_y-q_y)^2}$
$\rightsquigarrow$ Nachteil: Wurzel!
$\Rightarrow$ Besser: Vergleiche Quadrate der Distanzen:
$(p_x-q_x)^2+(p_y-q_y)^2 \overset{?}{\lessgtr} (p_x-r_x)^2+(p_y-r_y)^2$?
($\rightsquigarrow$ 4 Subtraktionen, 4 Multiplikationen, 2 Additionen).

1.2.7. Übertragung auf höhere Dimensionen:

Das Orientierungsprädikat kann auf höhere Dimensionen verallgemeinert werden.
Insbesondere im $\mathbb{R}^3$: seien $a,b,c,d\in\mathbb{R}^3$
orientation$(a,b,c,d)\in\{-1,0,1\}$
Sei orientation$(a,b,c,d)\neq 0$, betrachte Ebene durch a,b,c.



1.2.7.1 Satz: Für $a,b,c,d\in\mathbb{R}^3$ betrachte Ebene durch a,b,c.
Falls orientation $\neq 0$, so gilt:
(i) orientation$(a,b,c,d)=-1$, wenn von d aus $\Delta a,b,c$ negativ orientiert ist.
orientation$(a,b,c,d)=+1$, wenn von d aus $\Delta a,b,c$ positiv orientiert ist.
(ii) $$\text{orientation}(a,b,c,d) = \operatorname{sign}\begin{vmatrix} a_{x_1} & a_{x_2} & a_{x_3} & 1 \\ b_{x_1} & b_{x_2} & b_{x_3} & 1 \\ c_{x_1} & c_{x_2} & c_{x_3} & 1 \\ d_{x_1} & d_{x_2} & d_{x_3} & 1 \end{vmatrix}$$

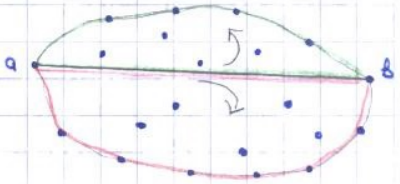
1.3. Algorithmus II: Graham's Scan.

1.3.1. Idee: Berechne "obere" und "untere" Hülle getrennt.

Sei a der linkeste und b der rechteste Pkt von S, dh. a = Minimum bzgl. x-y-Sortierung und b = Maximum bzgl. xy-Sortierung.

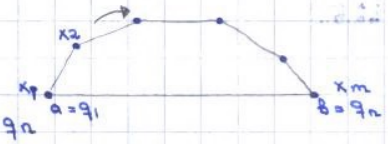
Obere Hülle = Der Teil von CH(S), der oberhalb von $\overline{ab}$ liegt (inclusive a,b). $= CH(S_1), S_1 \subset S$.

Untere Hülle = Der Teil von CH(S), der unterhalb von $\overline{ab}$ liegt (incl. a,b) $= CH(S_2), S_2 \subset S$.



Beobachtung: $S_1 = \{p \in S : \text{orientation}(a,b,p) \geq 0\}$
und $S_2 = \{p \in S : \text{orientation}(a,b,p) \leq 0\}$.

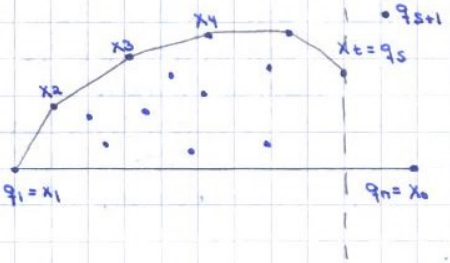
Sei S_1 lexikografisch nach xy-sortiert ($\to n \log n$)
$\rightsquigarrow$ sortierte Folge $q_1 \dots q_n = S_1$
$\Rightarrow a = q_1$ und $b = q_n$
Berechne obere Hülle als Teilfolge $x_1 .. x_m$ von $q_1 .. q_n$ (dh. im Uhrzeigersinn).



Beobachtung:
1). Jeweils drei aufeinanderfolgende Ecken x_i, x_{i+1}, x_{i+2} bilden einen Rightturn, dh. orientation $(x_i, x_{i+1}, x_{i+2}) < 0$.
2). (b,a,p) ist Rightturn $\forall p \in S_1$.
(dh. $\forall p \in S_1$: orientation $(q_n, q_1, p) < 0$).

1.3.2. Algorithmus:

verwalte stack $x_0, x_1 .. x_t$ von potentiellen Ecken der oberen Hülle.
Am Ende: genau die Ecken der oberen Hülle.
Initialisierung: stack $S = q_n, q_1, q_2$. Bem: alle drei Pkte wg. lexik. Sort. genau festgelegt und müssen nicht gesucht werden.
Schleife: Betrachte nur $q_3 .. q_{n-1}$.
Sei q_s aktueller Pkt
Invariante: $x_0, x_1 .. x_t$ ist Teilfolge von $q_n, q_1 .. q_s$



mit: 1) $t \geq 2$, $x_0 = q_n$, $x_1 = q_1$, $x_t = q_s$
besser x_1 statt x_0 2) $x_0 x_1 .. x_t$ ist konvexes Polygon
3) obere Hülle von S ist Teilfolge von $x_1 .. x_t\, q_{s+1} .. q_n$.

Schritt: $s \mapsto s+1$
while (x_{t-1}, x_t, q_{s+1}) nicht Rightturn do
pop (x_t)
od
push (q_{s+1}).

Beobachtung: stack speichert obere Hülle der Pkte $q_1 .. q_s$ (die bis jetzt gewesen).

Die Funktion:
UPPER_HULL $(q_1 .. q_n)$ besser: $q_1 .. q_m \Rightarrow b = q_m$ (weil $|S| = n$)
(Vorbed. 1). $q_1 .. q_n$ lexik. nach xy sortiert
2) $q_2 .. q_{n-1}$ liegen oberhalb $\overline{q_1 q_n}$).
Stack von Pkten. S
(stack <point> S_i)
S.push (q_n);
S.push (q_1);
S.push (q_2);
(Ann. $n \geq 2$).

→

```
for s=3 to n-1 do
    x ← S.top();          gibt das oberste Element an.
    y ← S.top_pred();     eins unter dem obersten auf dem Stack
    while orientation (y, x, q_s) ≥ 0 do
        S.pop();
        x ← y;
        y ← S.top_pred();
    od
    S.push (q_s)
od
return S.
```

While-Schleife terminiert spätestens, wenn S nur noch die Pkte q_n, q_1 enthält. (weil alle Pkte $q_2 .. q_{n-1}$ oberhalb von $\overline{q_1 q_n}$ liegen.).

1.3.3. Beispiel für die Pkt:



Anfang: Stack $q_6 q_1 q_2$.

s = 3

$x = q_2 \wedge y = q_1$

orientation $(q_1 q_2 q_3) = -1$

$\Rightarrow$ Stack $q_6 q_1 q_2 q_3$

s = 4

$x = q_3 \wedge y = q_2$.

orientation $(q_2 q_3, q_4) = -1 \Rightarrow$ Stack $q_6 q_1 q_2 q_3 q_4$

s = 5

$x = q_4 \wedge y = q_3$

orientation $(q_3 q_4 q_5) = +1 \Rightarrow$ stack $q_6 q_1 q_2 q_3$, $x = q_3, y = q_2$

orientation $(q_2 q_3 q_5) = -1 \Rightarrow$ stack $q_6 q_1 q_2 q_3 q_5$

fertig!

1.3.4. Korrektheit.

(1) Invariante ist stets erfüllt.

Zeige mit Induktion, dass Invariante stets erfüllt ist.

Ind.anf: $S = q_n q_1 q_2$

$x_0, x_1, x_2 = q_n q_1 q_2$ mit 1) $t = 2, q_n = x_0, q_1 = x_1, q_2 = x_2$

2) $x_0 x_1 x_2$ konv. Polygon

3) obere Hülle von S ist TF von $x_0 x_1 x_2 q_3 \ldots q_{n-1}$.

Ind.ann: die Invariante sei für $x_0 x_1 .. x_t$ erfüllt $(t \in \mathbb{N})$.

Indbeh: Sei q_s der nächste Pkt bzgl. der lex.kogr. Sortierung.

Wenn orientation $(x_{t-1}, x_t, q_s) = -1$ dann ist die Beh. trivialerweise erfüllt.

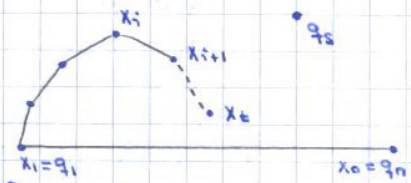
Sei also orientation $(x_{t-1}, x_t q_s) \in \{1, 0\}$

z.z. $x_0 \ldots x_t$ ist TF von $q_n q_1 .. q_s$ mit

1) $t \geq 2, x_0 = q_n, x_1 = q_1, x_t = q_s$

2) $x_0 .. x_t$ konv. Polygon

3) obere Hülle von S ist TF von $x_1 .. x_t q_{s+1} .. q_n$.



$\Rightarrow S = x_0 x_1 .. x_{t-1}$

überall x_1. ... wird so lange gepopt bis orientation $(y, x, q_s) < 0$.

$\Rightarrow S = x_0 x_1 .. x_i$

S.push $(q_s) \Rightarrow S = x_0 x_1 .. x_i x_t$, wobei $x_t := q_s$

$\Rightarrow x_0 x_1 .. x_i x_t$ ist TF von $q_n q_1 \ldots q_s$

zu 1) $\underbrace{t \geq 2}$, $x_0 = q_n, x_1 = q_1$ und $x_t = q_s$ nach Def. $\Rightarrow$ ok.
weil $(x_0 x_1 q_s)$ Rightturn $\Rightarrow i \geq 1 \Rightarrow t \geq 2$.

zu 2). Folge $x_0 .. x_i$ unverändert und (x_{i-1}, x_i, q_s) bilden Rightturn

$\Rightarrow x_0 .. x_i x_t$ auch konvexes Polygon.

zu 3) Pkte $x_{i+1} .. x_{t-1}$ wurden entfernt. Sie gehören nicht zu oberen Hülle, da sie rechts von (oder auf) Strecke $\overline{x_i q_s}$ liegen. $\Rightarrow$ ok.

$\Rightarrow$ Beh.

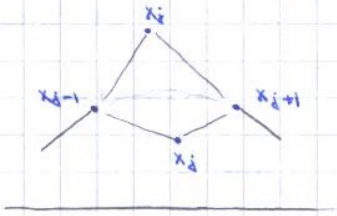
(2). Bei Termination enthält Stack S die obere Hülle.

Voraussetzung: die Invariante ist für den letzten Pkt ($s=n$) erfüllt.

d.h. alle Pkte sind abgearbeitet.

Wissen also: $x_1 .. x_t$ ist TF von $q_1 .. q_n$ mit bsp: $t \geq 2$, $x_0 = q_n$, $x_1 = q_1$ und $x_t = q_n$

1) $t \geq 2$, $x_0 = q_n$, $x_1 = q_1$ und $x_t = q_n$
2) $x_1 .. x_t$ konv. Polygon
3). obere Hülle ist TF von $x_1 .. x_t$.



z.Z. obere Hülle $= x_1 .. x_t$ also nicht echte Teilmenge.

Ann: nein, obere Hülle $\subset x_1 .. x_t$

$\Rightarrow \exists\, x_j \in \{x_1 .. x_t\}$ mit $x_j \notin$ obere Hülle

$\Rightarrow$ 1.Fall: orientation $(x_{j-1}, x_j, x_{j+1}) = +1 \Rightarrow$ ↯ zu 2)

2.Fall: orientation $(x_{j-1}, x_j, x_{j+1}) = -1 \Rightarrow$ ↯ zu 2).

$\Rightarrow$ Ann. falsch

$\Rightarrow$ Beh.

1.3.5. Laufzeit:

Beobachtung: Jeder Pkt kann höchstens einmal aus S entfernt werden.

Rumpf hat Kosten $O(1)$ (orientation, konstante Zahl von stack-Operationen).

Da jeder Pkt höchstens einmal aus S entfernt werden kann $\Rightarrow$

$\Rightarrow$ while-Schleife wird höchstens n mal eingetreten } $\Rightarrow$ forschleife kostet uns $O(n)$.

In der while-Schleife konst. Operationen, außerhalb auch }

$\Rightarrow$ Gesamtkosten der UPPER-Pkt : $O(n)$.

(Analog berechnet man die untere Hülle).

Die Funktion insgesamt:

```
CONVEX_HULL(S)
1. S.sort(xy);                                   O(n log n)   (|S| = n)
   a ← S.min();                                  O(1)
   b ← S.max();                                  O(1)
2. forall p ∈ S do   eigentlich S \ {a,b}
3.     if (orientation(a,b,p) > 0)  S1.append(p)   O(n)      (S1,S2 Listen ⇒ append kostet
4.     if (orientation(a,b,p) < 0)  S2.append(p)              O(1), n x ⇒ O(n))
5. od
6. S1.push(a);                                   O(n)        (falls S1 Liste ⇒
   S1.append(b);                                 O(1)         einfügen am Ende kostet O(n))
7. S2.push(a);                                   O(n)
   S2.append(b);                                 O(1)
8. H1 ← UPPER_HULL(S1)                           O(m) = O(n)
9. H2 ← LOWER_HULL(S2)                           O(n-m) = O(n)
10. H1 umdrehen                                  O(n)
11. H2 an H1 anhängen                            O(1)
12 doppeltes Vorkommen von a/b löschen.          O(n)
```

Zusammenfassung:

CONVEX_HULL(S)	Zeit:
1) Sortiere S lexik.	$O(n \log n)$
2) $a \leftarrow \min \wedge b \leftarrow \max$	$O(1)$
3) $S_1 \leftarrow \{p \in S : b,a,p$ Rechtsturm $\} \cup \{a,b\}$	$O(n)$
4) $S_2 \leftarrow \{p \in S : b,a,p$ Linksturm $\} \cup \{a,b\}$	$O(n)$
5) UPPER_HULL(S_1)	$O(n)$
6) LOWER_HULL(S_2)	$O(n)$
7) " Zusammenkleben	$O(n)$